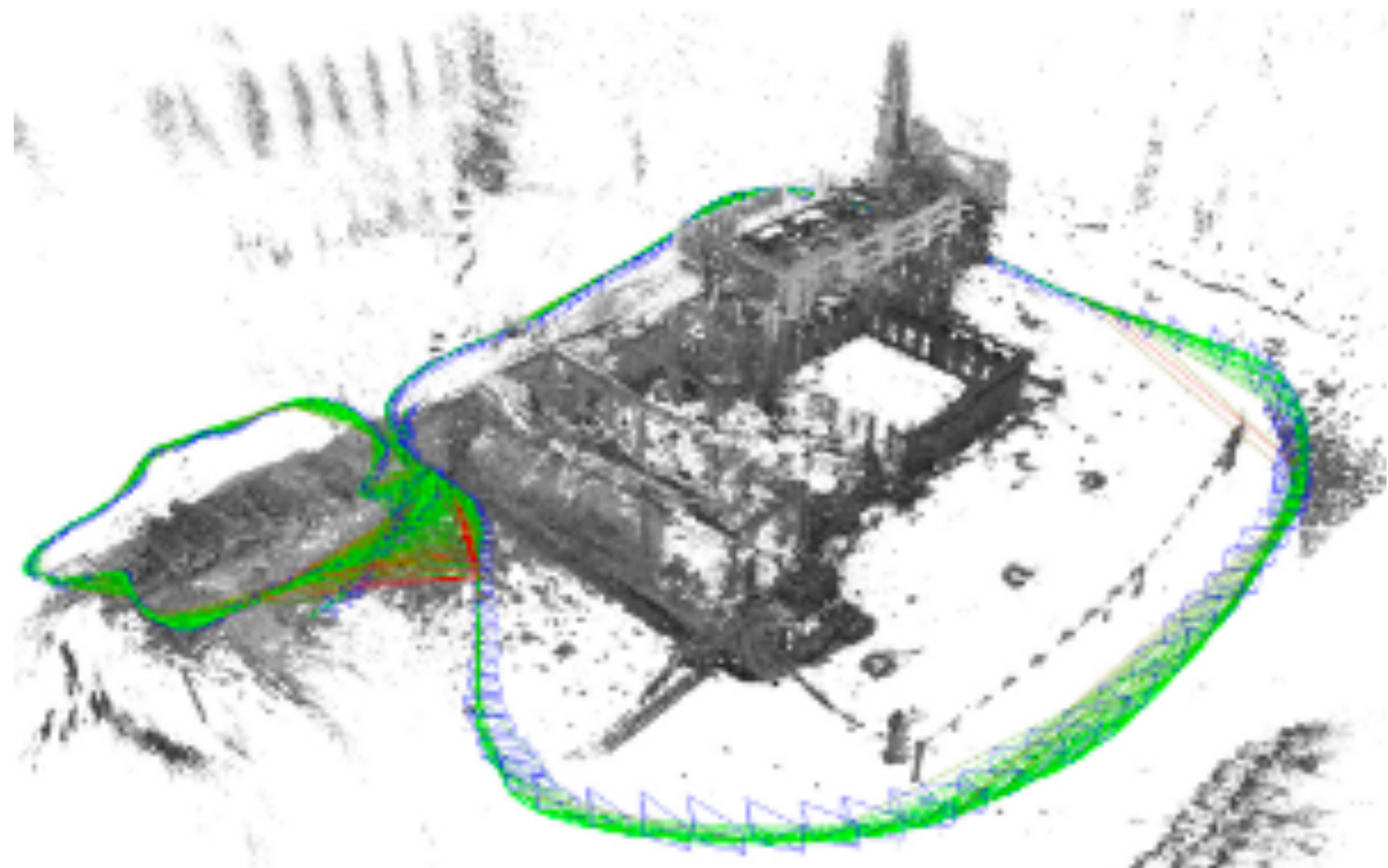


Geometry-based Methods in Vision

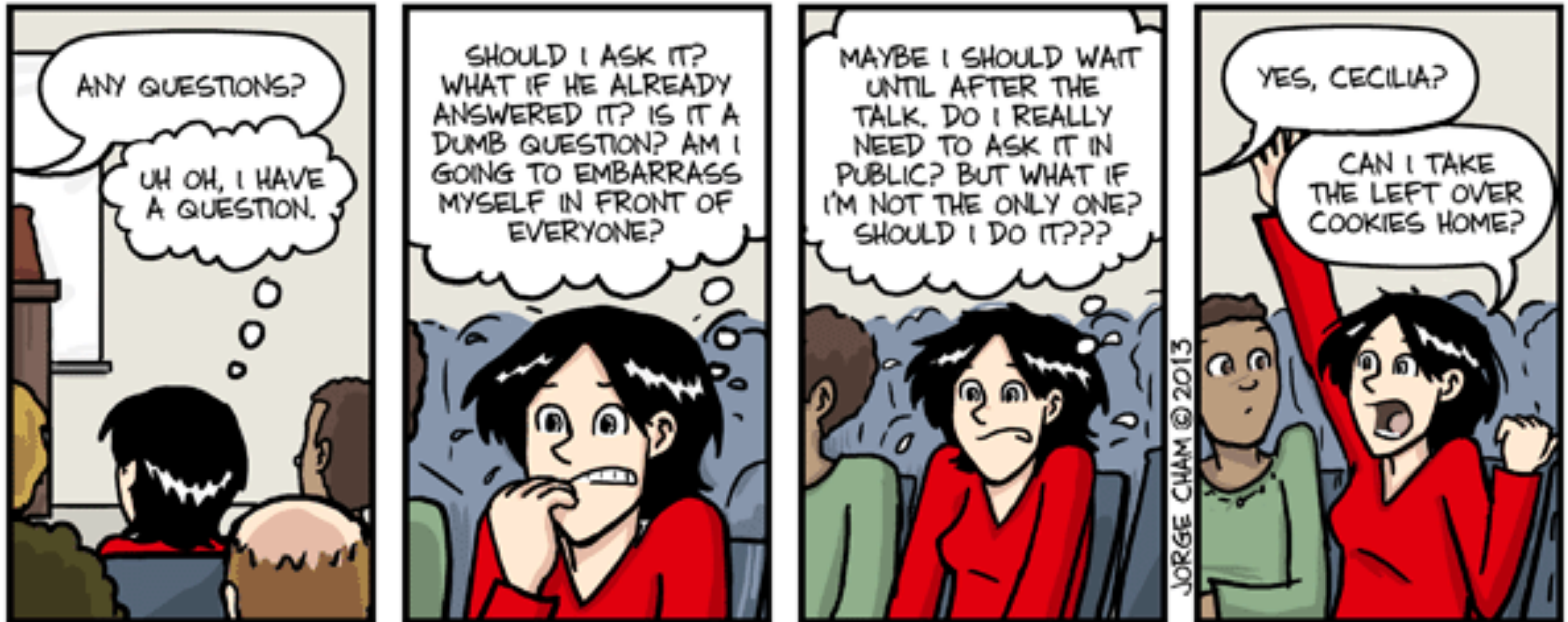
CMU 16-822, Fall 2024

Instructor: Shubham Tulsiani

Lecture 21: Simultaneous Localization and Mapping (SLAM)



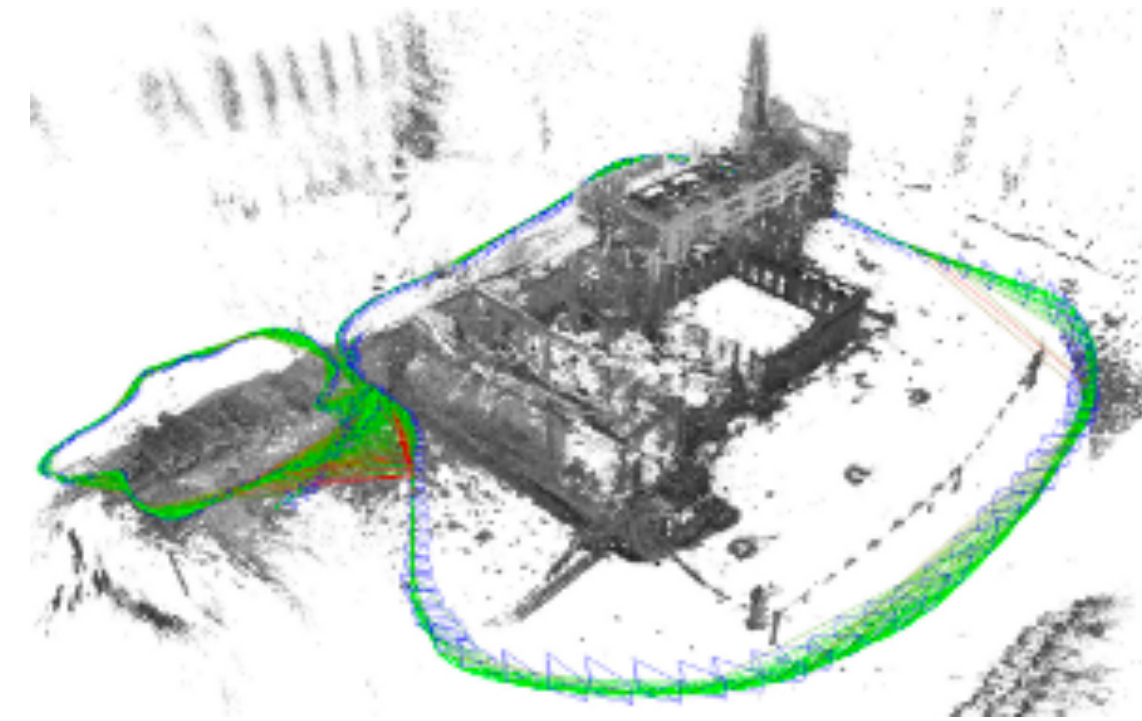
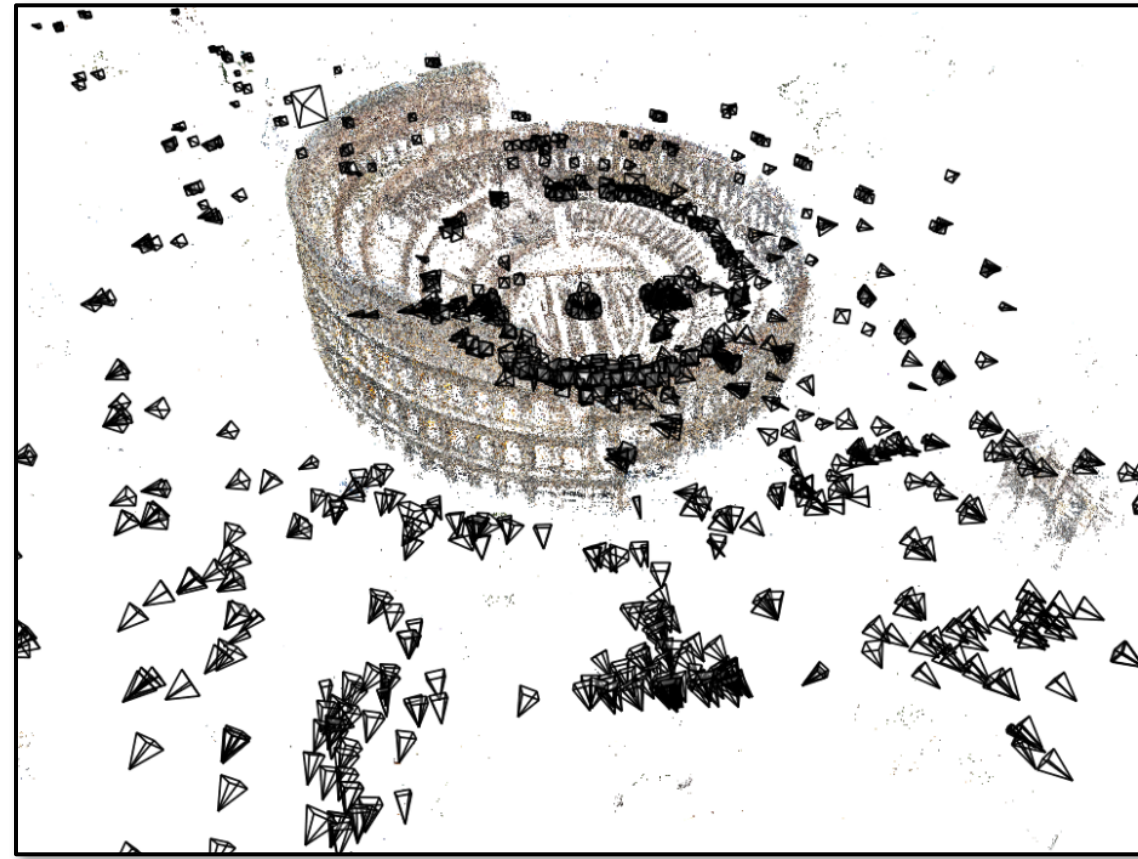
A reminder to ask questions!





Video Credit: DroidSLAM. *Teed et. al.*

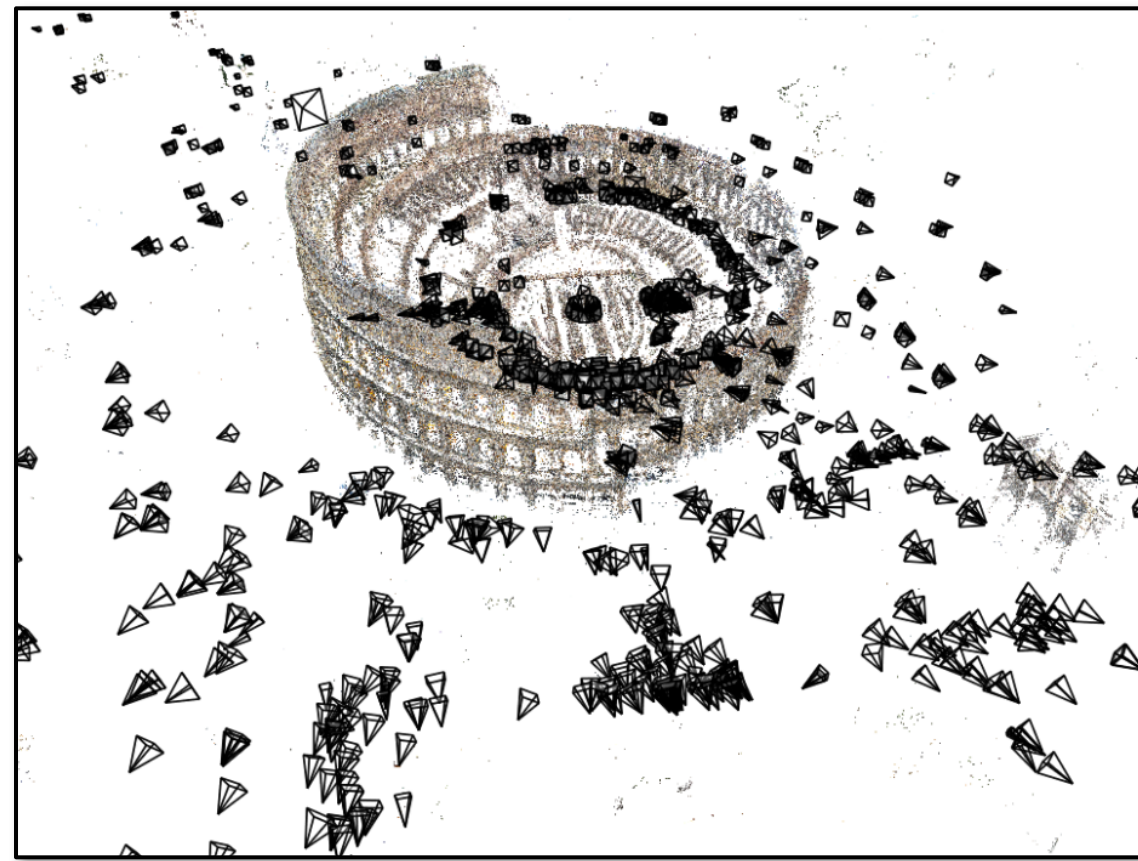
SfM and SLAM: Similarities



Goal: Recover camera poses and 3d points given images

Correspondence, registration, outlier rejection, and bundle adjustment are core problems

SfM and SLAM: Differences

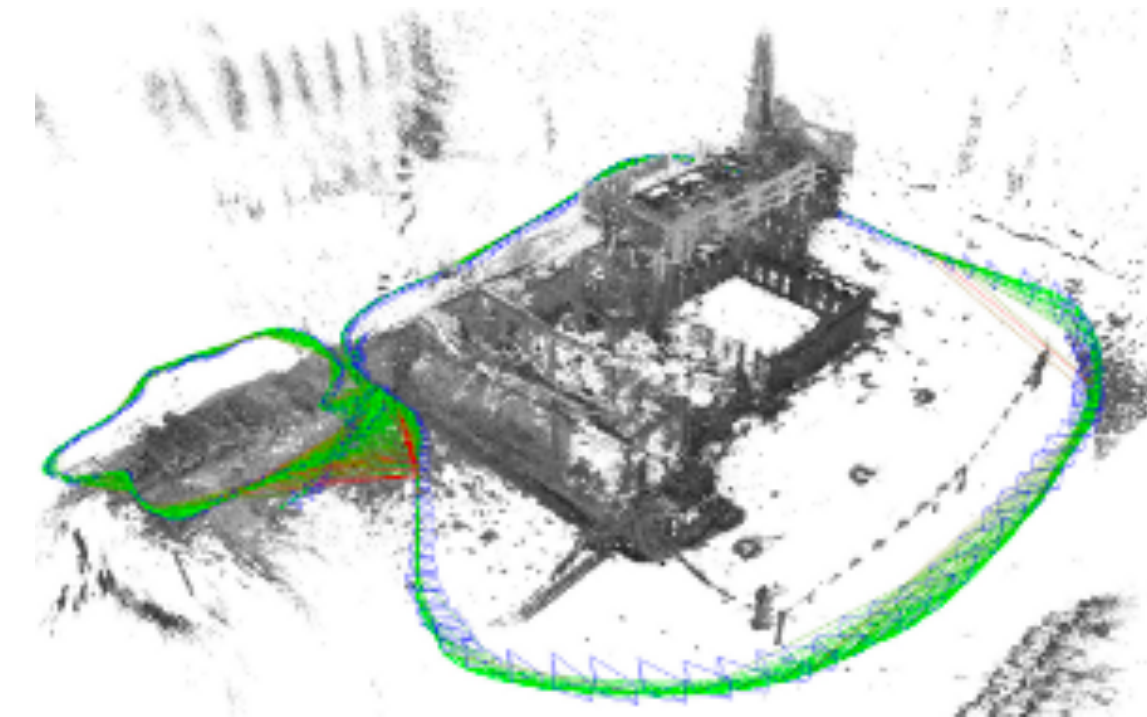


Input: **Unordered** set of images

Offline processing typically suffices

Complicated

(typical for real-world vision systems)



Input: **Sequence** of images

Online processing needed for
real-world applications

Very complicated

(typical for real-world robotics systems)

Overview

Simultaneous:

Localization

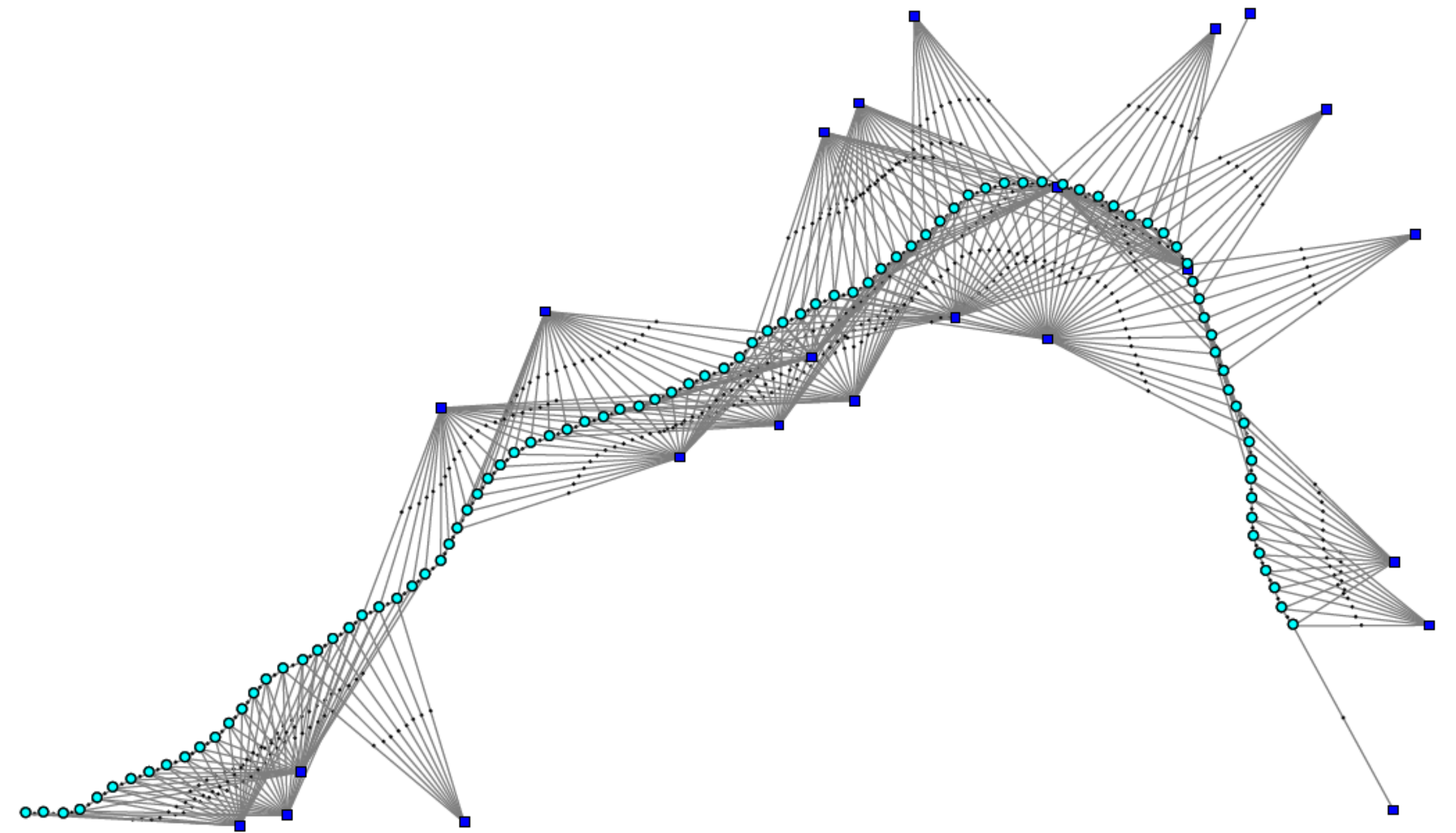
Estimation of state of the agent i.e.
position/orientation

Localization done w.r.t. the map

Mapping

Constructing a 3D representation of the
environment

Map is continuously expanded and refined



Jing Dong "[GTSAM 4.0 Tutorial](#)" License CC BY-NC-SA 3.0

Overview

Simultaneous:

Localization

Estimation of state of the agent i.e.
position/orientation

Localization done w.r.t. the map

Mapping

Constructing a 3D representation of the
environment

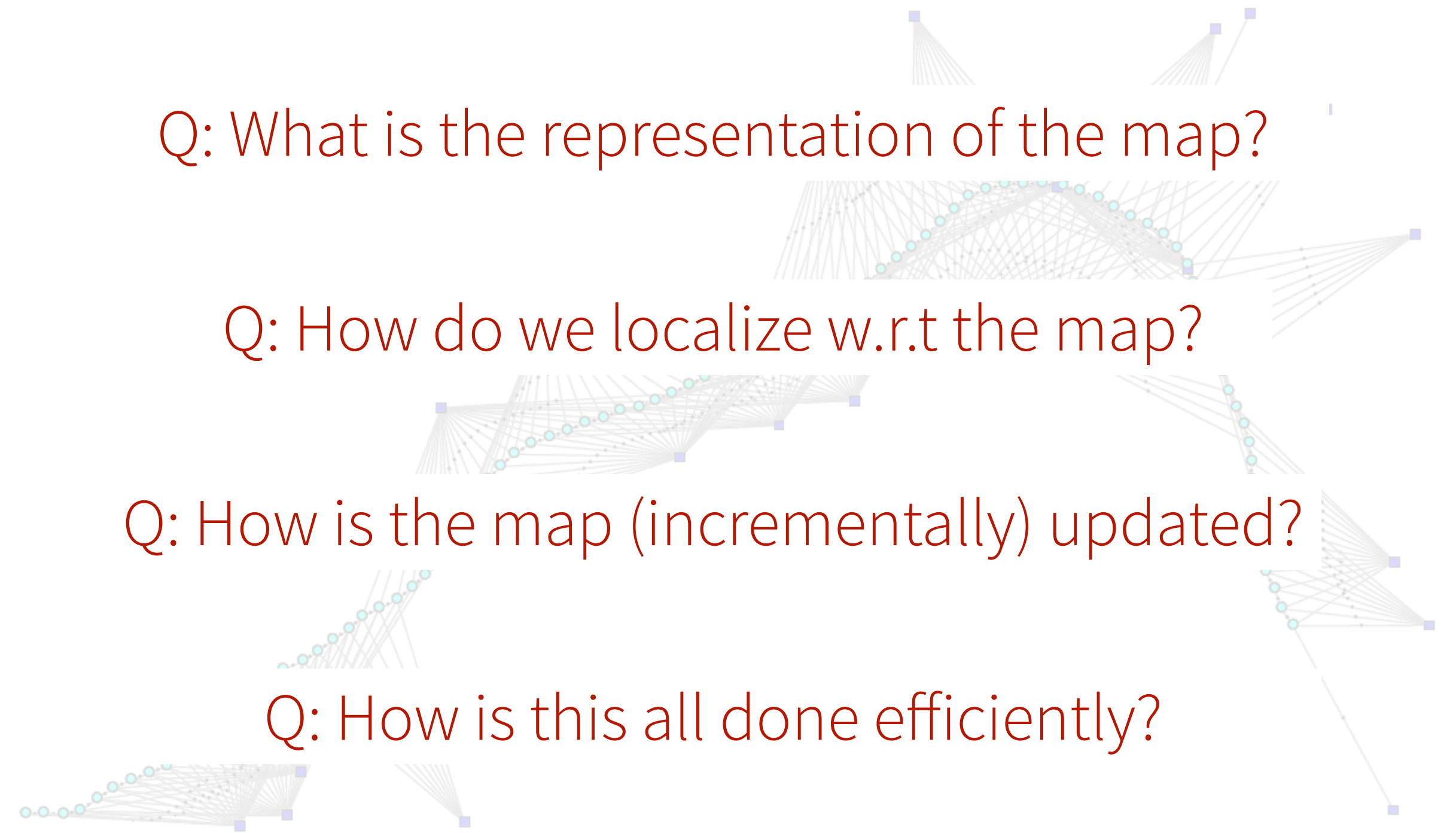
Map is continuously expanded and refined

Q: What is the representation of the map?

Q: How do we localize w.r.t the map?

Q: How is the map (incrementally) updated?

Q: How is this all done efficiently?



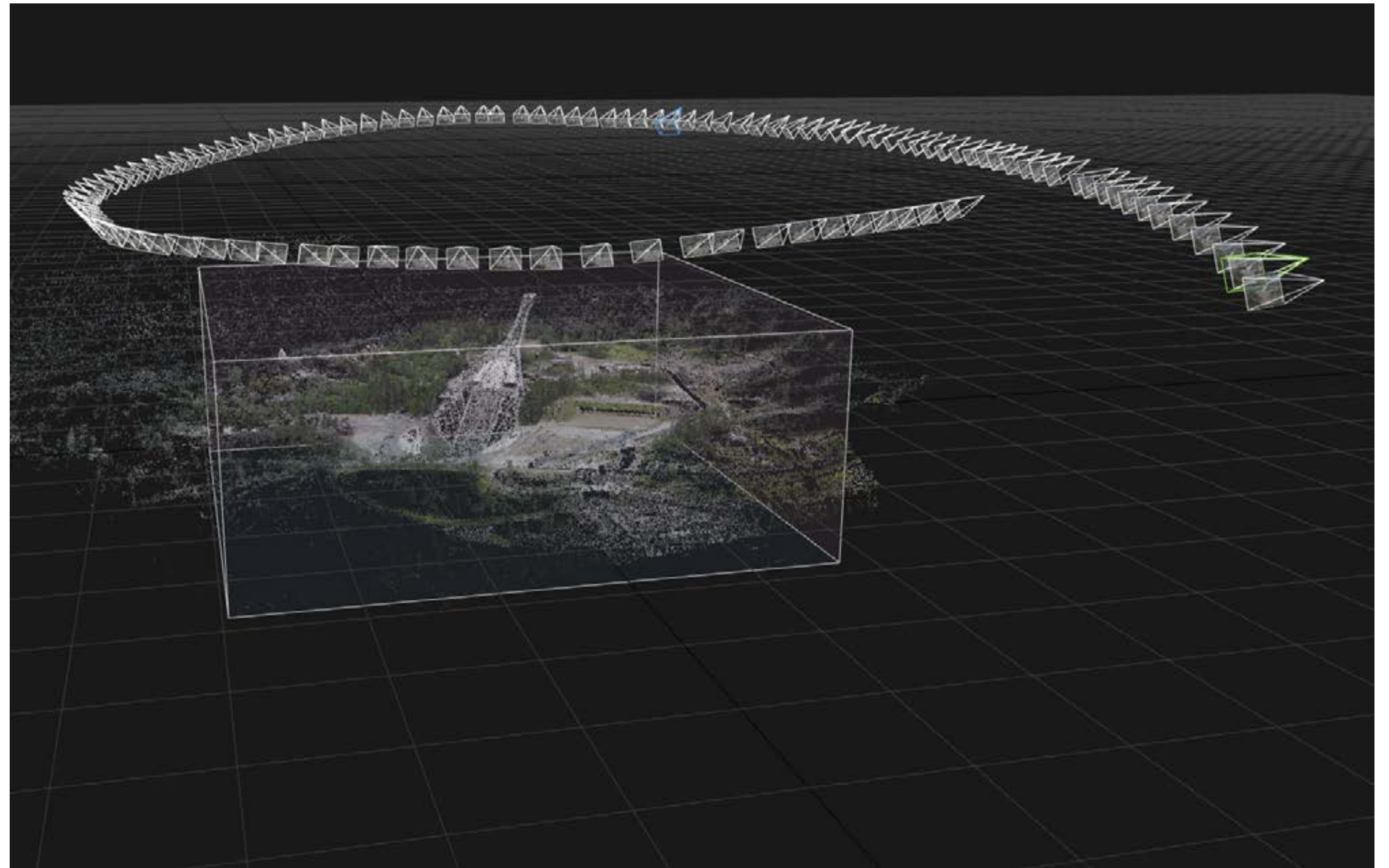
Jing Dong "[GTSAM 4.0 Tutorial](#)" License CC BY-NC-SA 3.0

SLAM: Map Representations

Q: What is the role of a map?

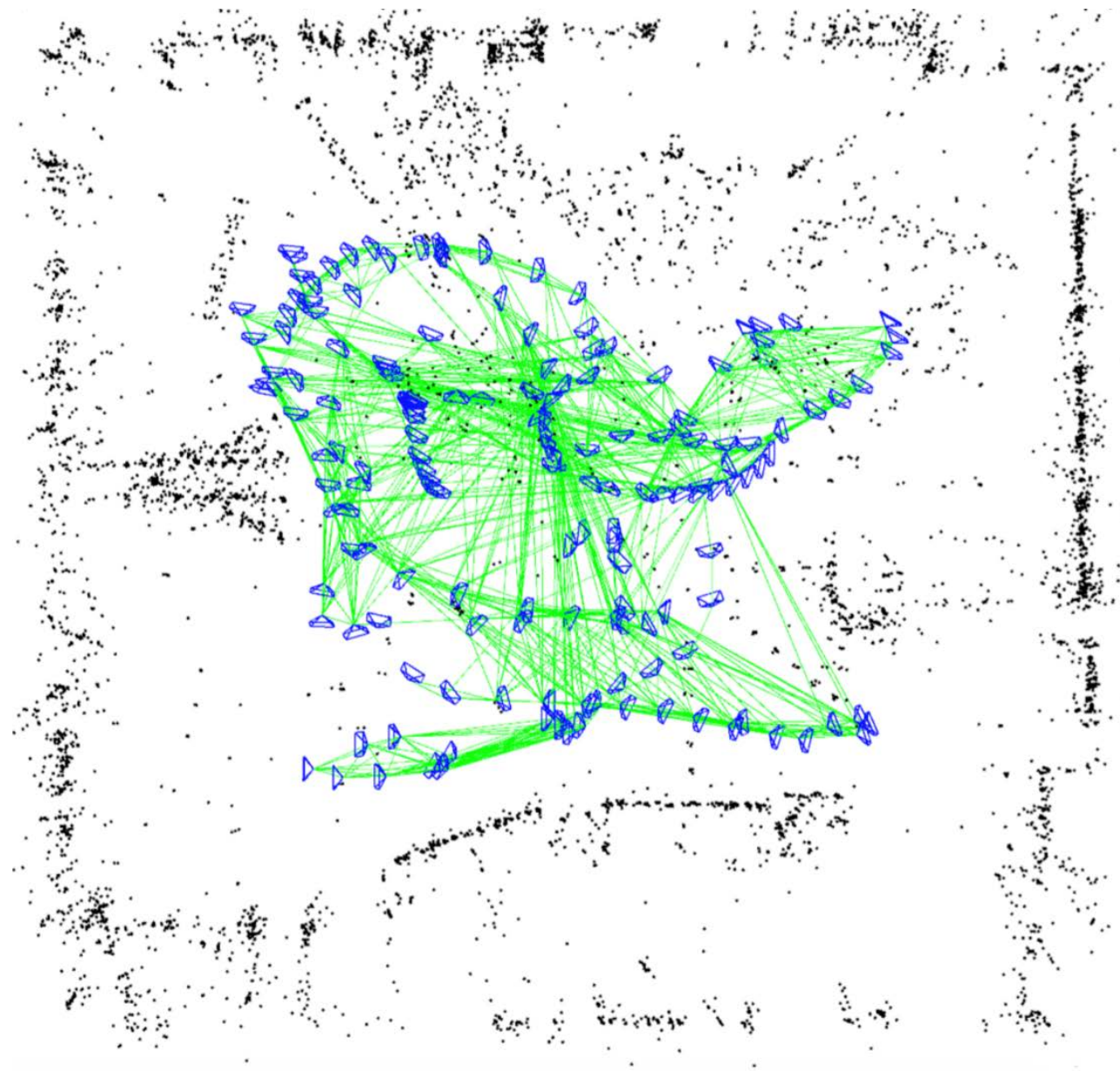
A: To help localization!

(often, we might want to do other things e.g. path planning, guided exploration etc. — but typically done as post-processing)



Map: Any representation that allows localization

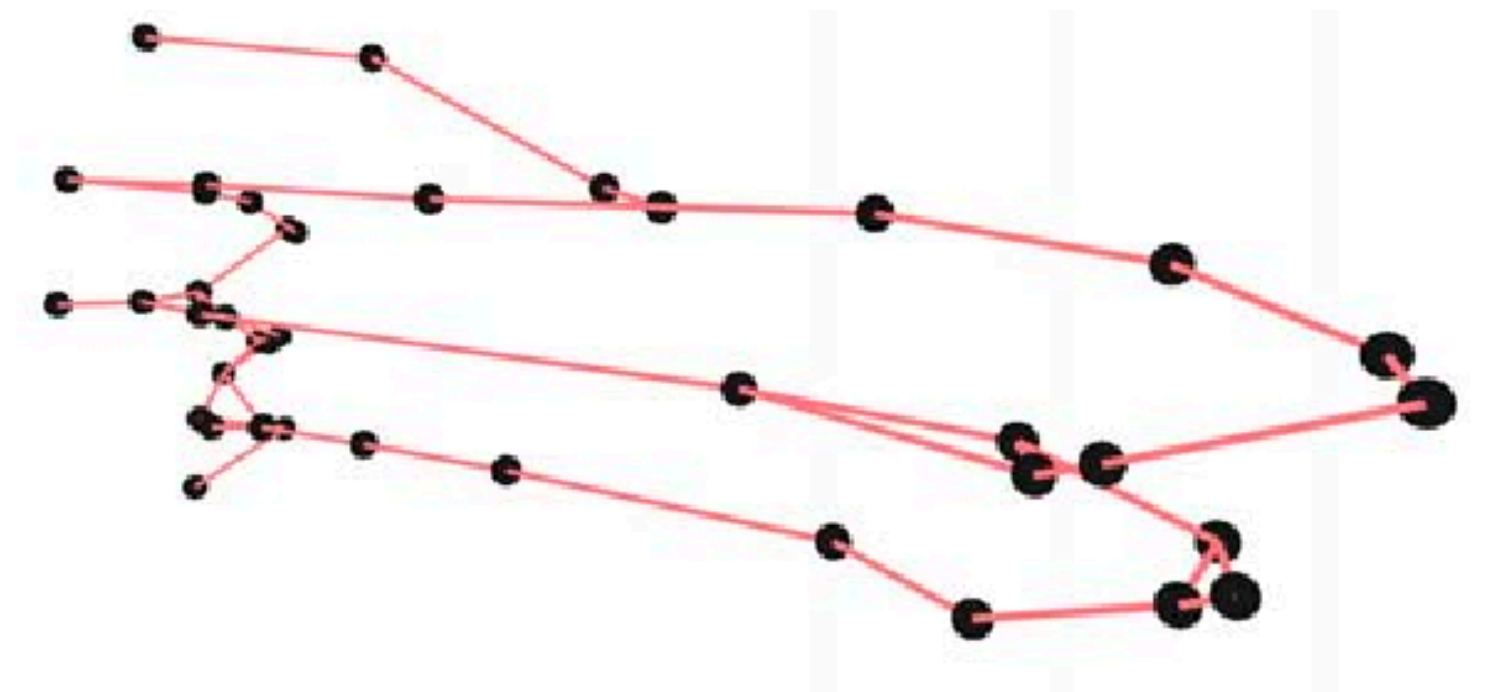
SLAM: Map Representations



Feature-based Metric Maps (typically sparse)



Dense Metric Maps



Topological Maps

(arguably closer to what humans represent)

Overview

Simultaneous:

Localization

Estimation of state of the agent i.e.
position/orientation

Localization done w.r.t. the map

Mapping

Constructing a 3D representation of the
environment

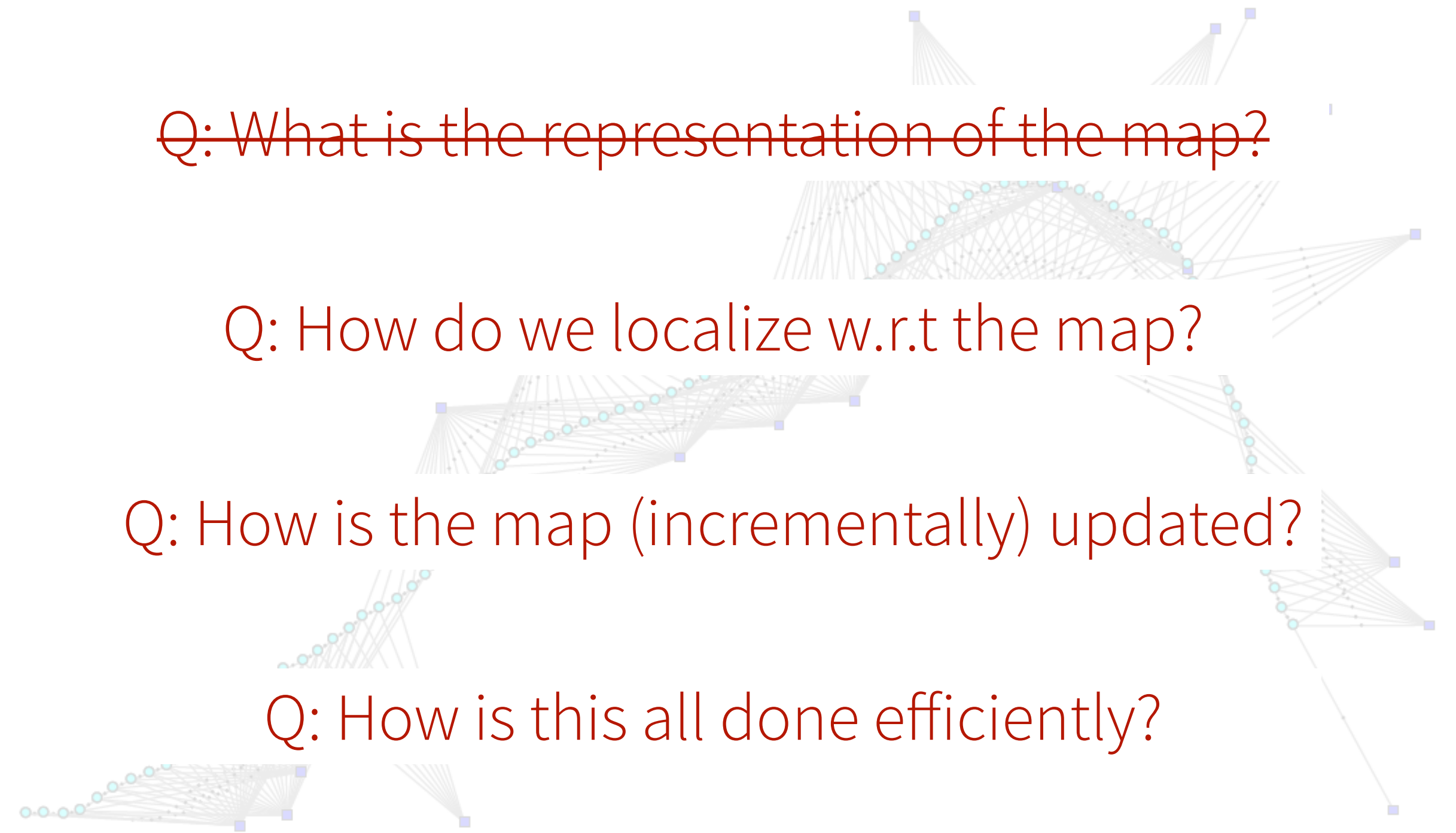
Map is continuously expanded and refined

~~Q: What is the representation of the map?~~

Q: How do we localize w.r.t the map?

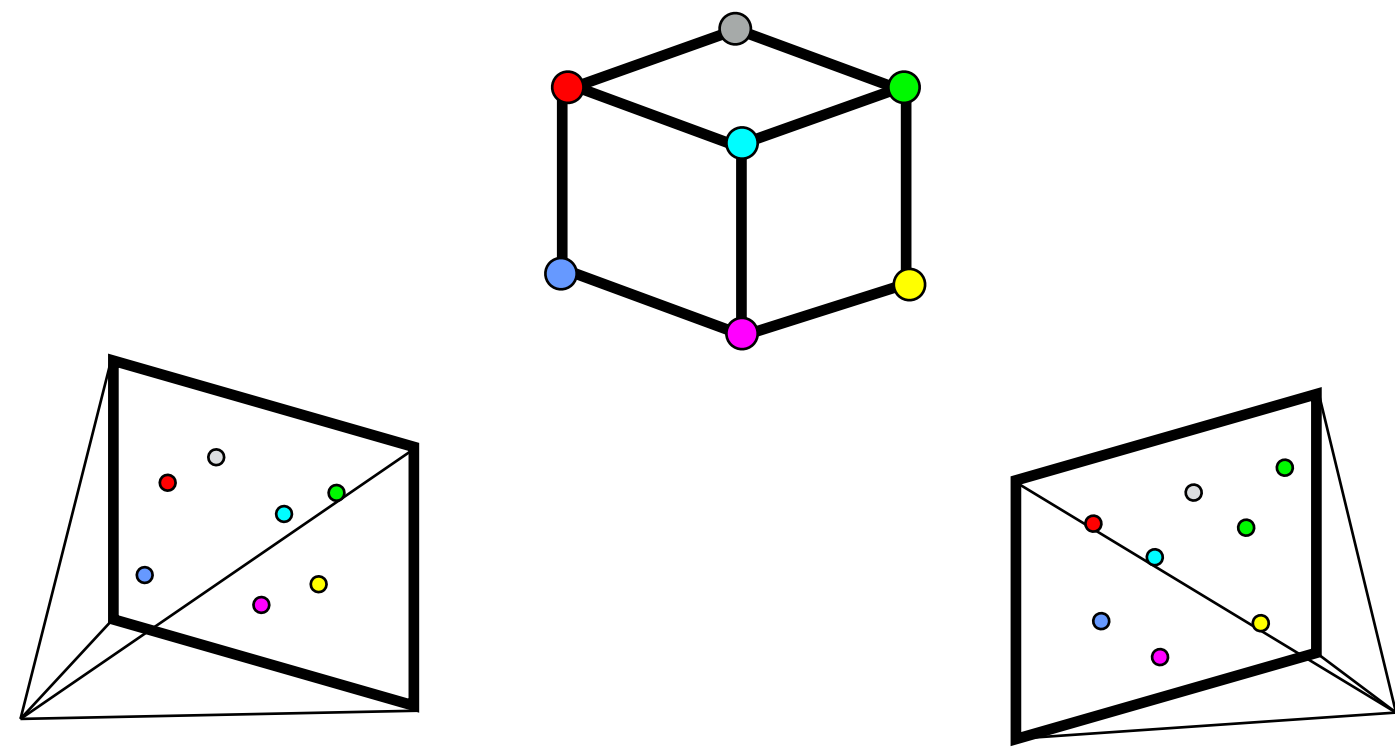
Q: How is the map (incrementally) updated?

Q: How is this all done efficiently?

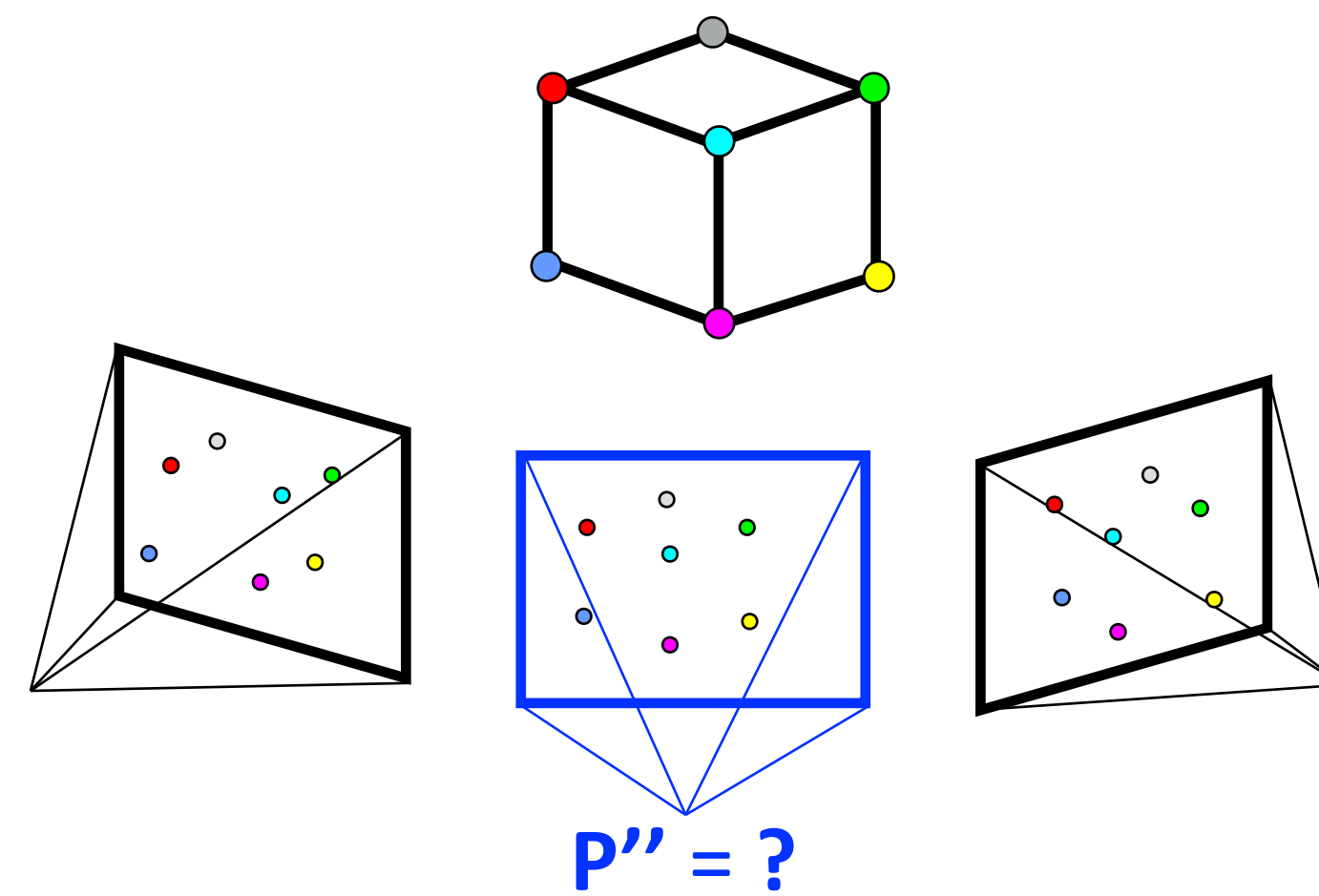


Jing Dong "[GTSAM 4.0 Tutorial](#)" License CC BY-NC-SA 3.0

Let's build a (SfM-inspired) SLAM System!

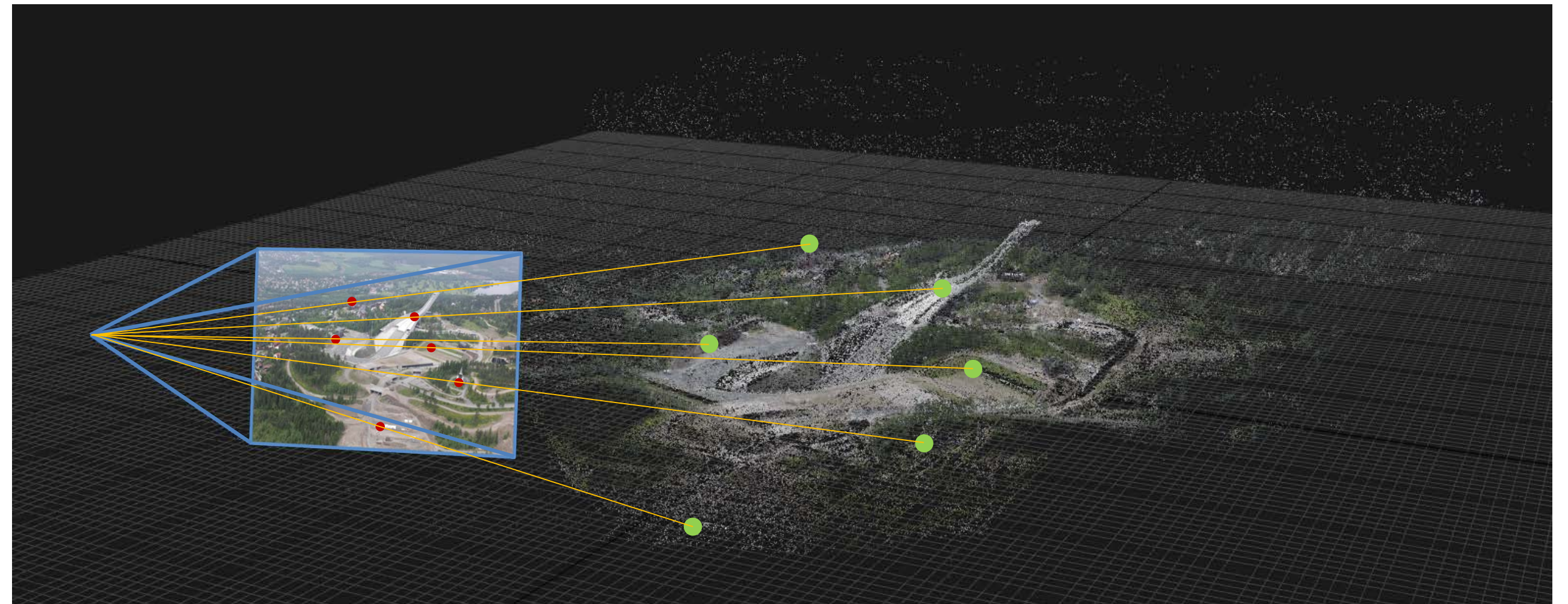
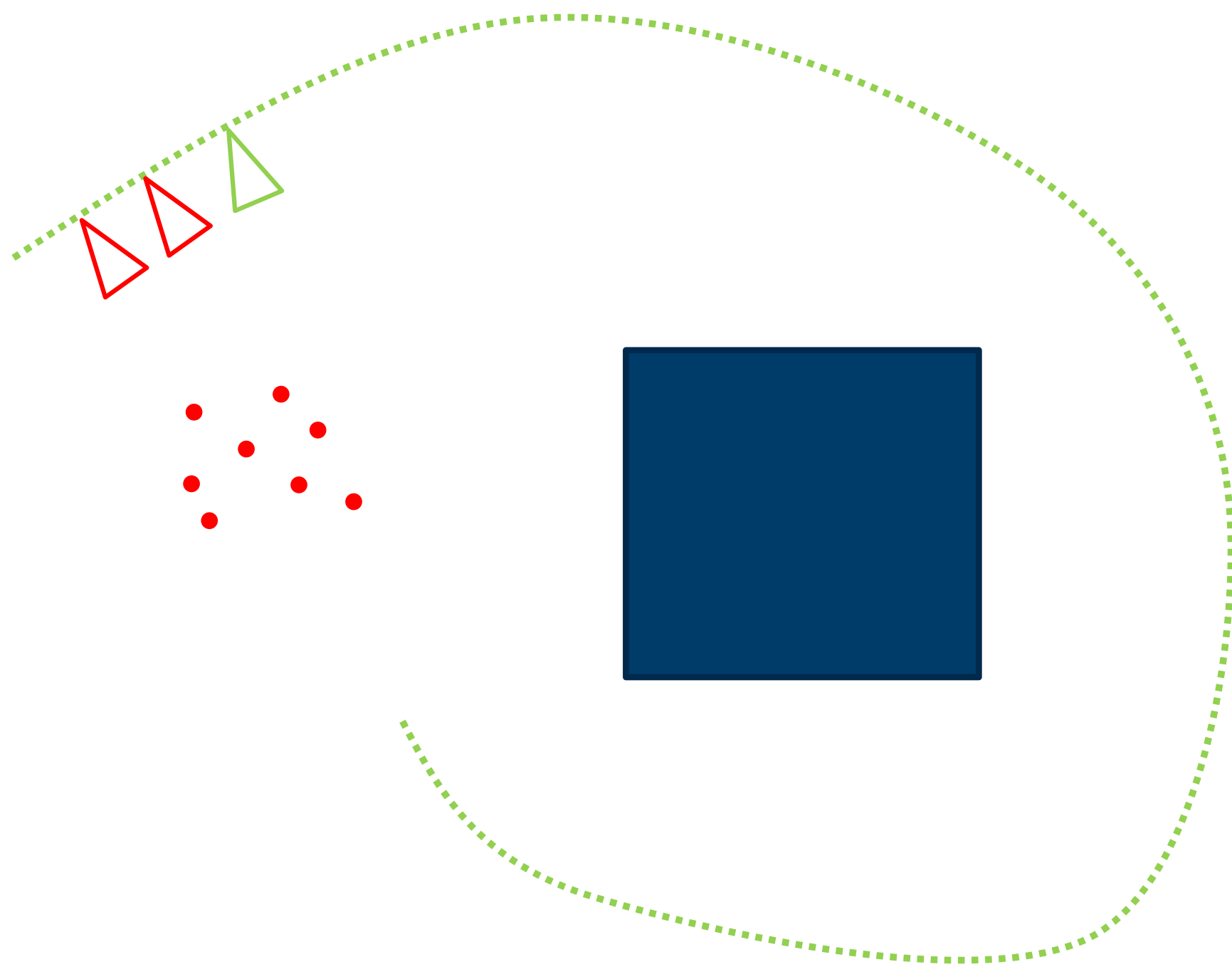


Mapping



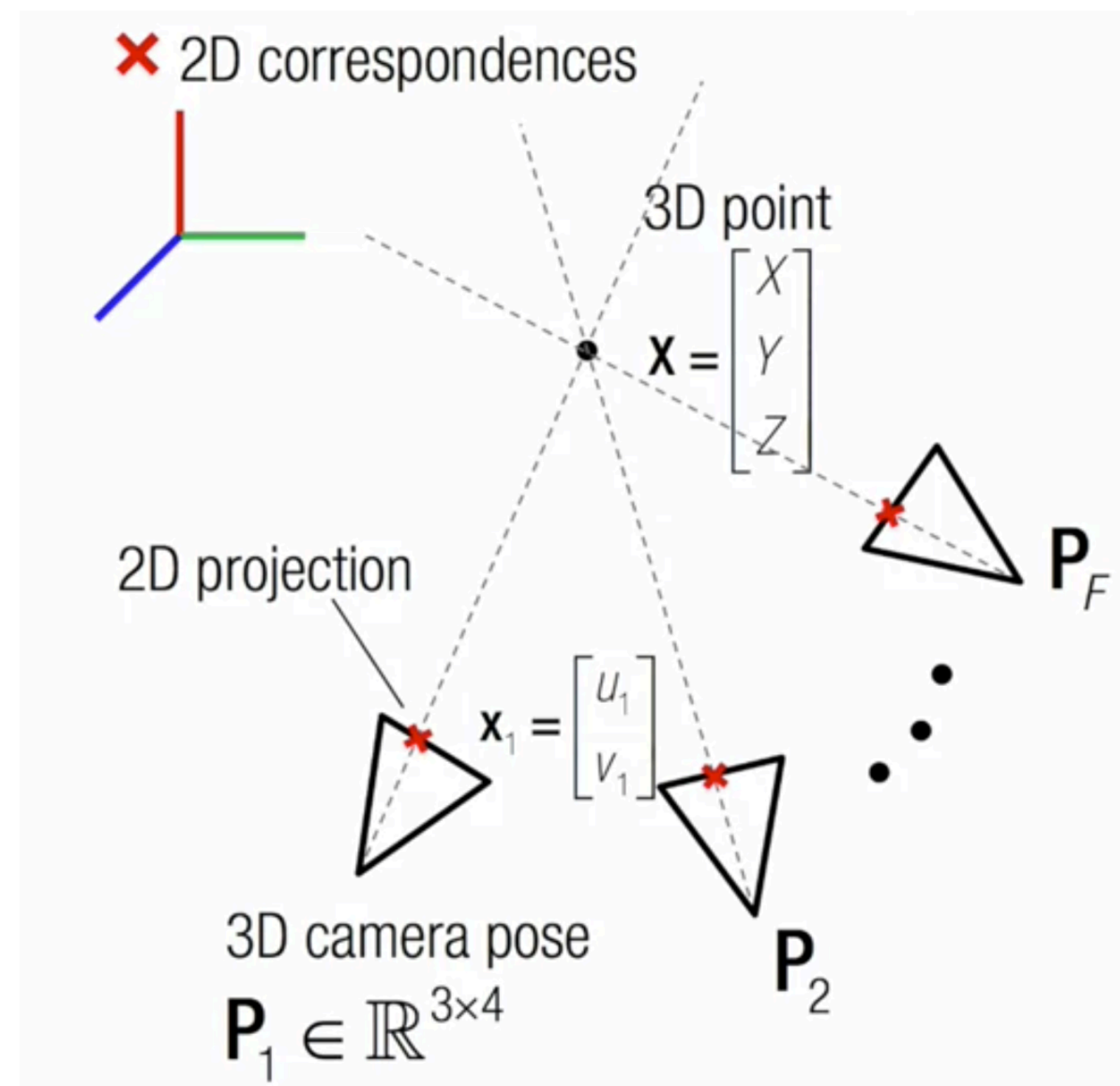
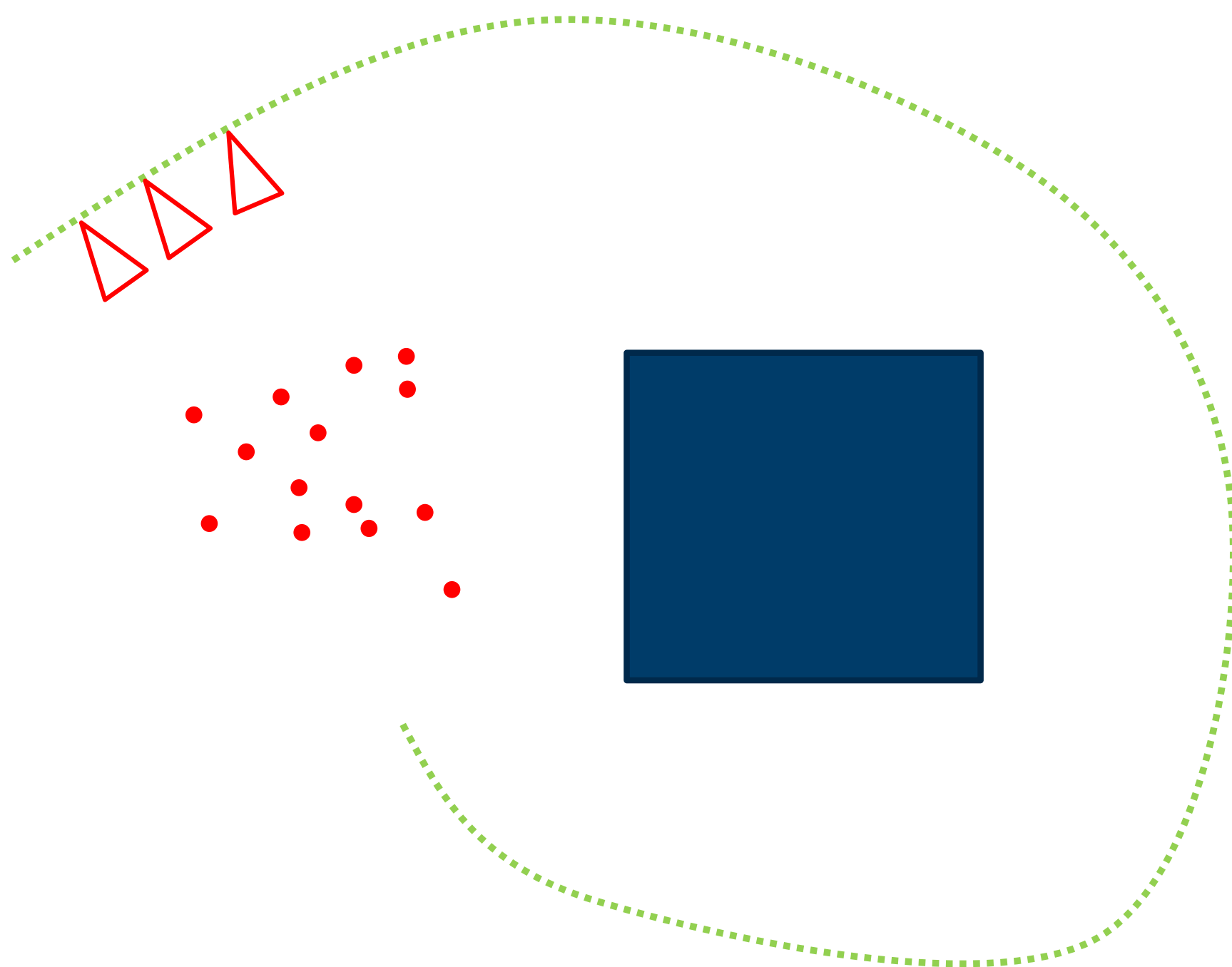
Localization

Let's build a SLAM System: Localization



$$\mathbf{T}_{wc}^* = \operatorname{argmin}_{\mathbf{T}_{wc}} \sum_i \left\| \pi(\mathbf{T}_{wc}^{-1} \cdot \mathbf{x}_i^w) - \mathbf{u}_i \right\|^2$$

Let's build a SLAM System: Mapping

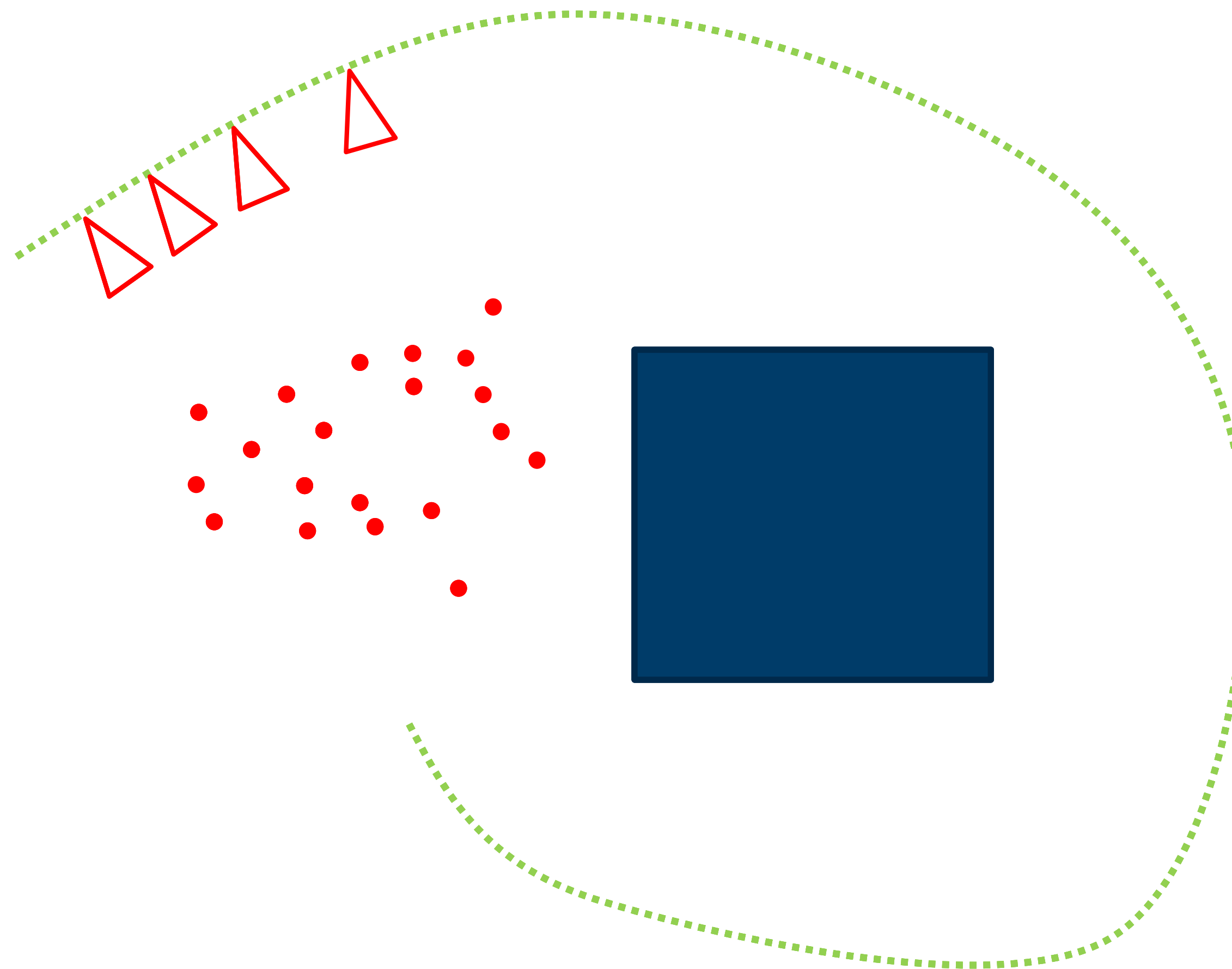


$$\begin{aligned}
 E_{\text{repro}} &= (u - u_{\text{repro}})^2 + (v - v_{\text{repro}})^2 \\
 &= \left(u - \frac{\mathbf{P}_1^T \tilde{\mathbf{X}}}{\mathbf{P}_1^3 \tilde{\mathbf{X}}} \right)^2 + \left(v - \frac{\mathbf{P}_2^T \tilde{\mathbf{X}}}{\mathbf{P}_2^3 \tilde{\mathbf{X}}} \right)^2
 \end{aligned}$$

$$\min_{\mathbf{x}} \sum_{i=1}^F \left(u_i - \frac{\mathbf{P}_i^T \tilde{\mathbf{X}}}{\mathbf{P}_i^3 \tilde{\mathbf{X}}} \right)^2 + \left(v_i - \frac{\mathbf{P}_i^T \tilde{\mathbf{X}}}{\mathbf{P}_i^3 \tilde{\mathbf{X}}} \right)^2$$

Reprojection error minimization

Let's build a SLAM System



And let's keep adding more frames...

Repeat:

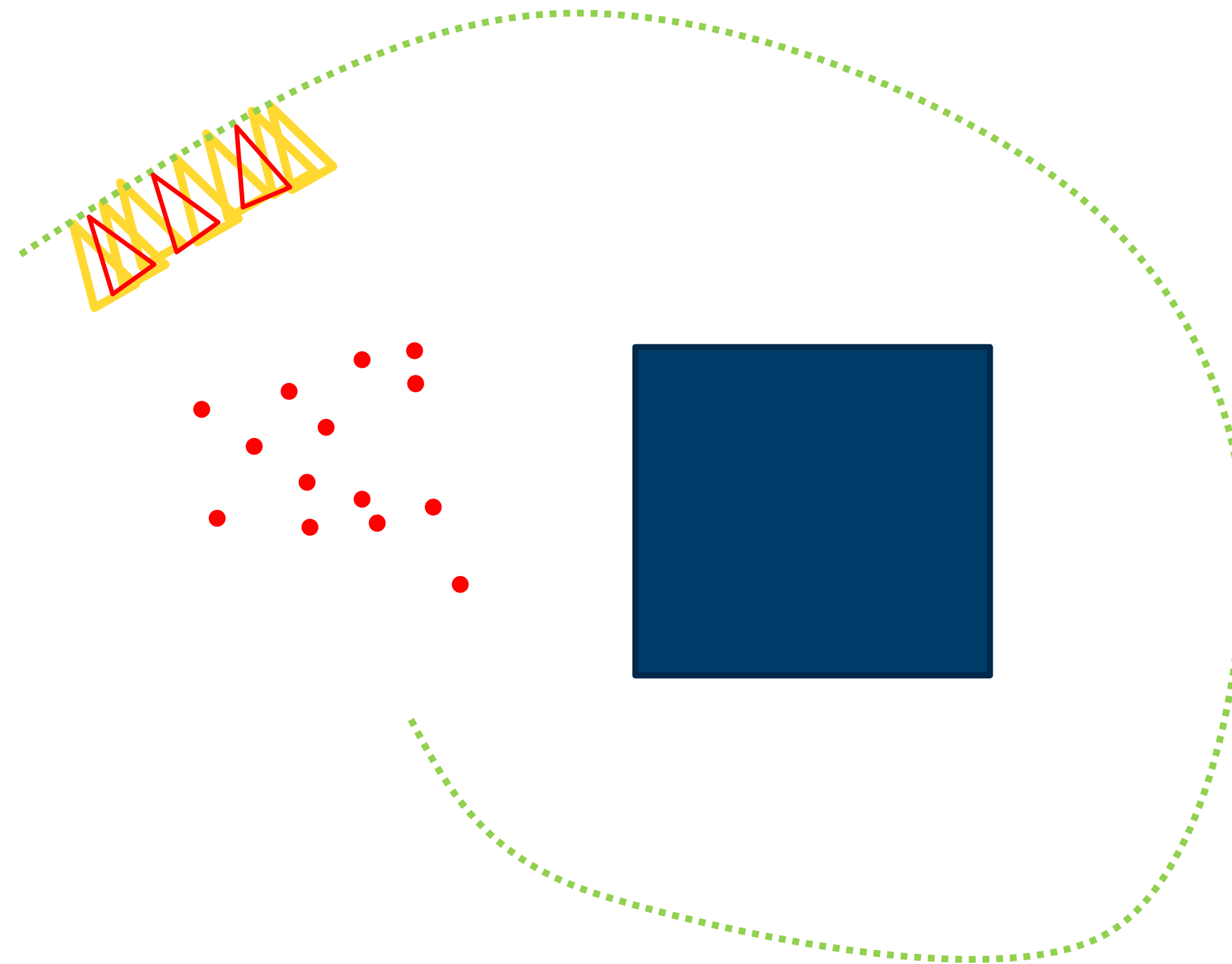
- a) Localize new frame
- b) Add new points to map
- c) Bundle-adjust poses & points

Q: what could be a bottleneck in doing this?

Compute intensive as more points/frames added!

Aspects of a Practical Solution: Keyframes

We have **lots** of input frames in a video (~30fps)!



Solution: only use some 'keyframes' for creating the map
(can localize all frames though)

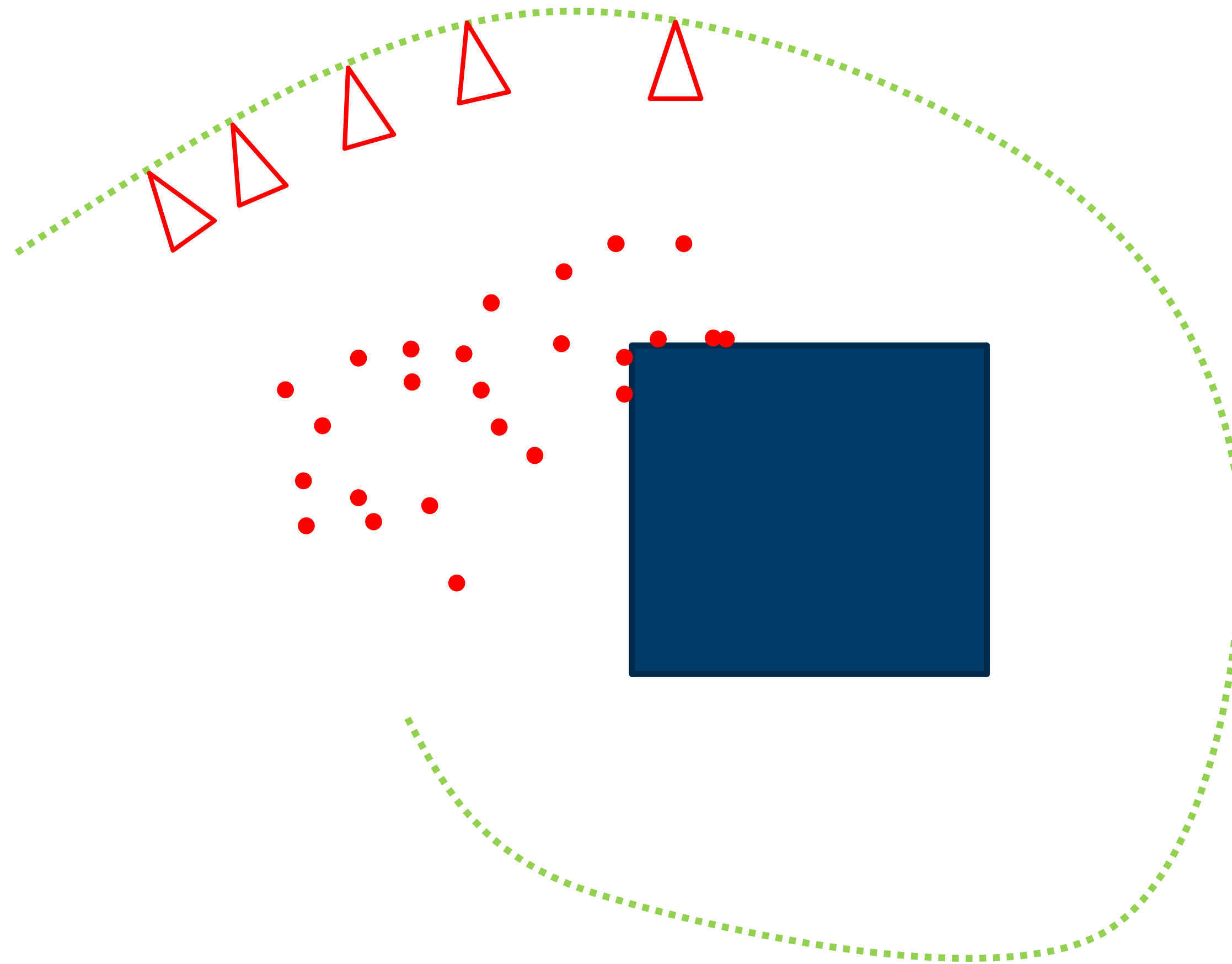
Aspects of a Practical Solution: Keyframes

Sample rules for KeyFrame insertion (from ORBSLAM)

Add current frame as new keyframe K_i if all conditions are met:

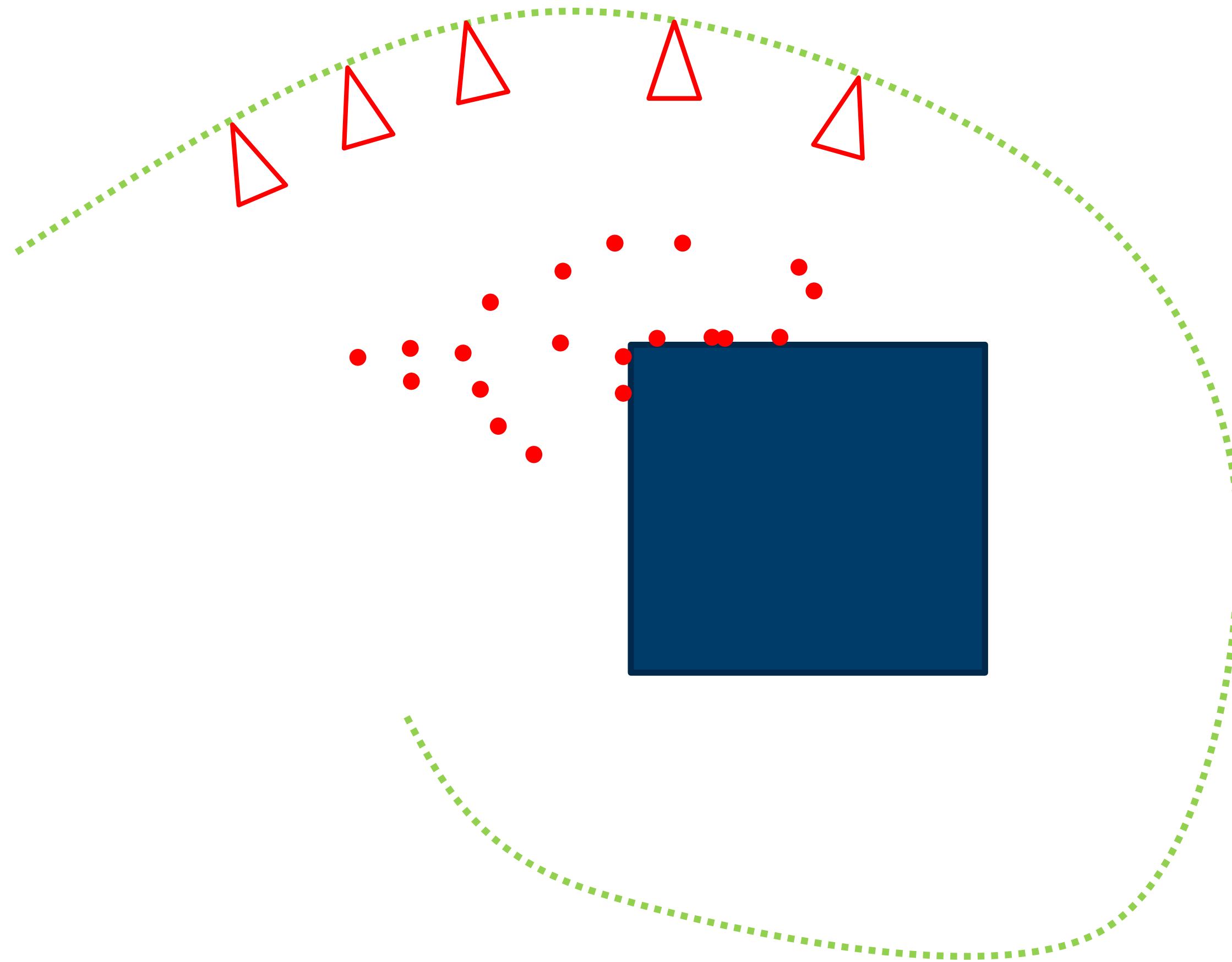
1. More than 20 frames since last global relocalization
2. Ready for new: Local mapping idle, or more than 20 frames since last keyframe insertion
3. Good tracking: Current frame tracks at least 50 points
4. Not redundant: Current frame tracks less than 90% of points from most similar keyframe

Aspects of a Practical Solution: Sliding Window Mapping



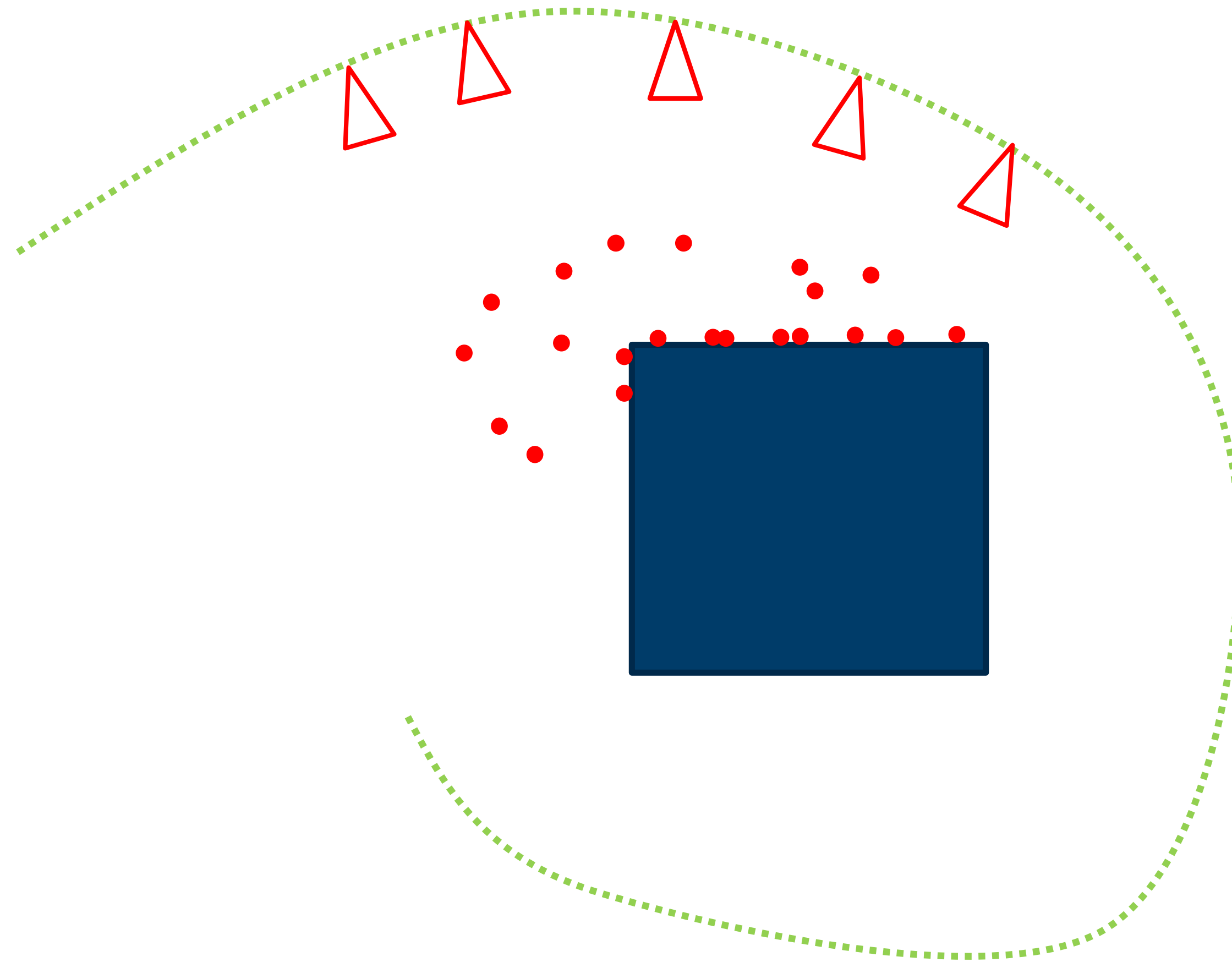
Even with keyframes, can have long sequences...

Aspects of a Practical Solution: Sliding Window Mapping

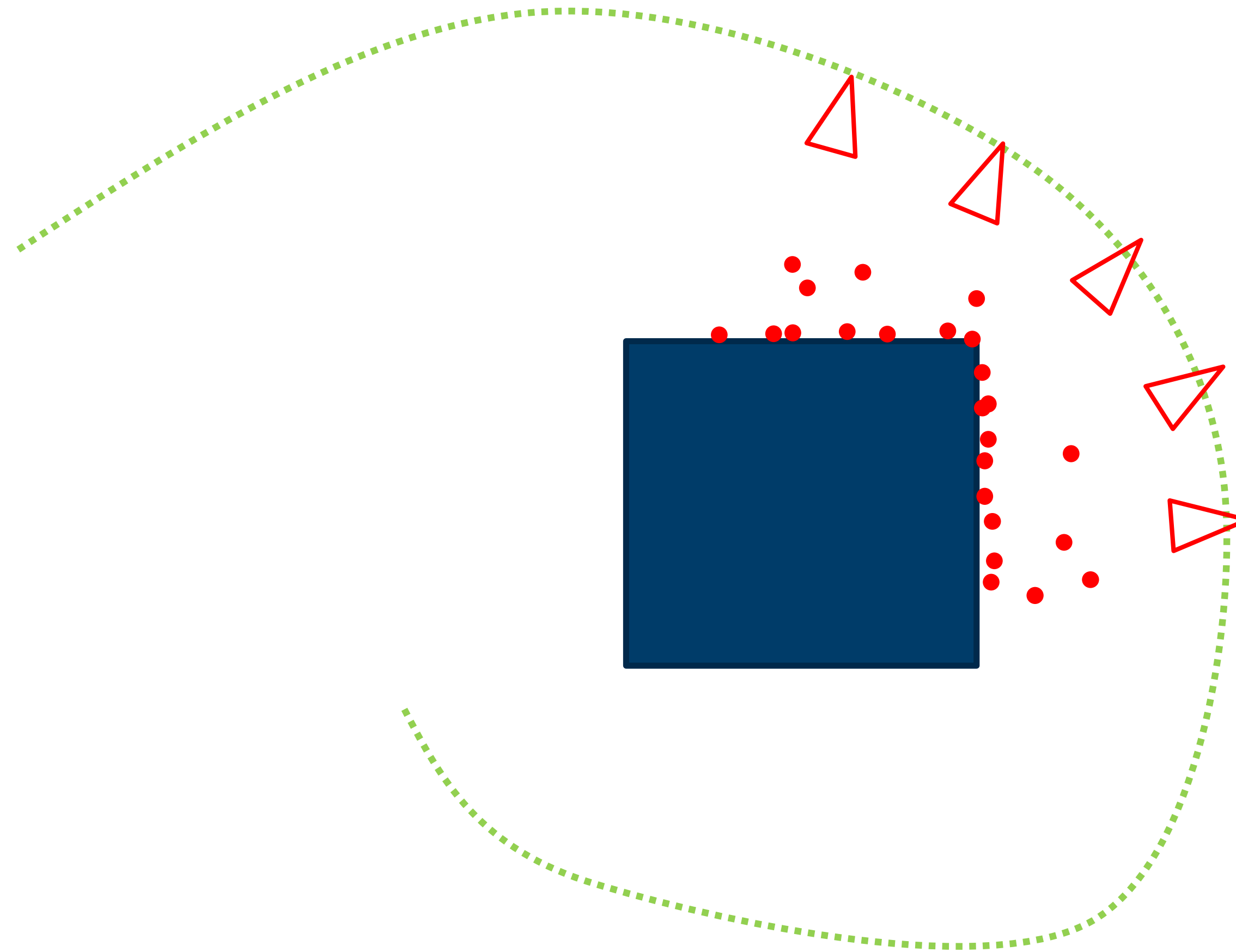


Only optimize poses (and points) for last few frames

Aspects of a Practical Solution: Sliding Window Mapping

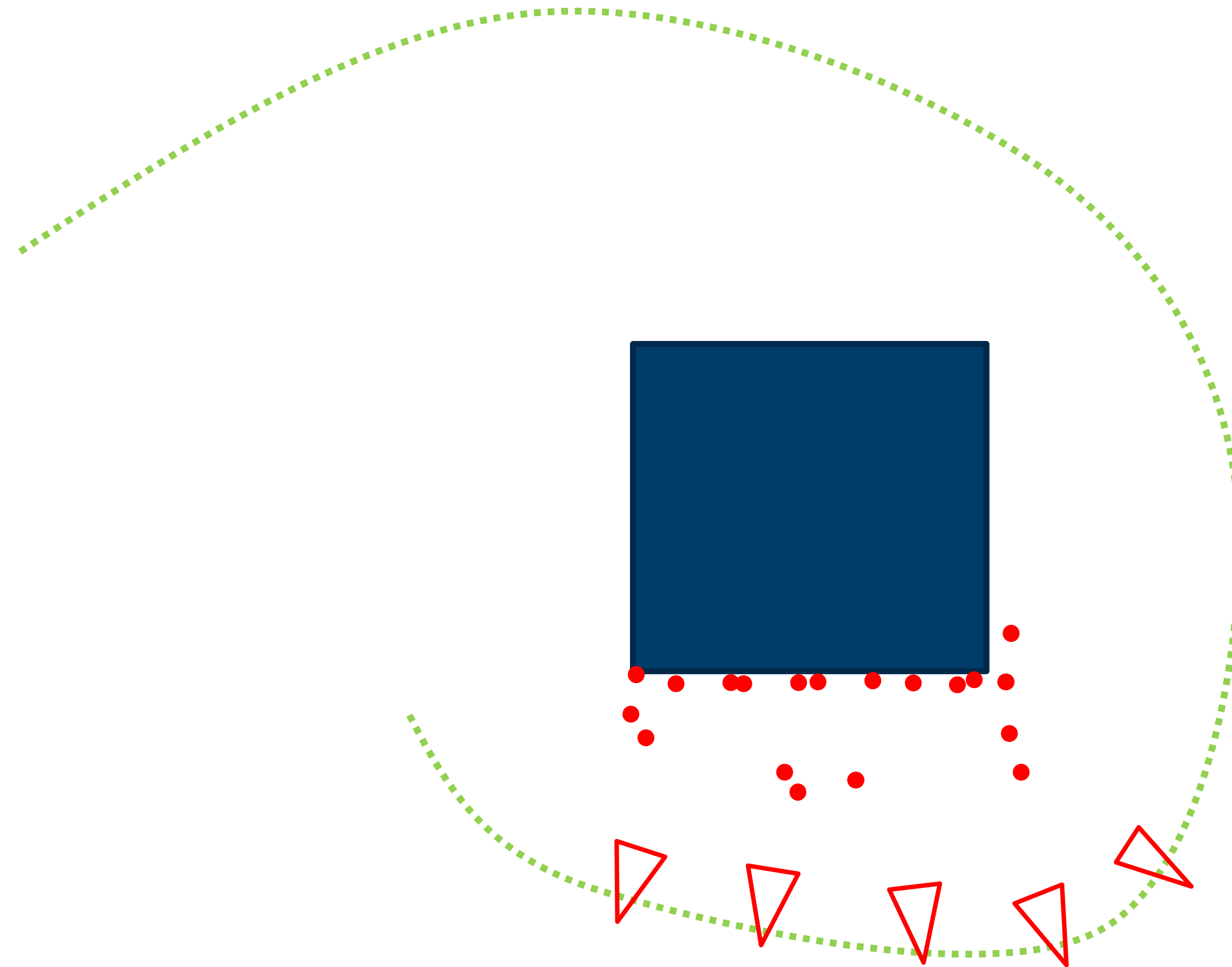


Aspects of a Practical Solution: Sliding Window Mapping



Errors can accumulate and cause drift

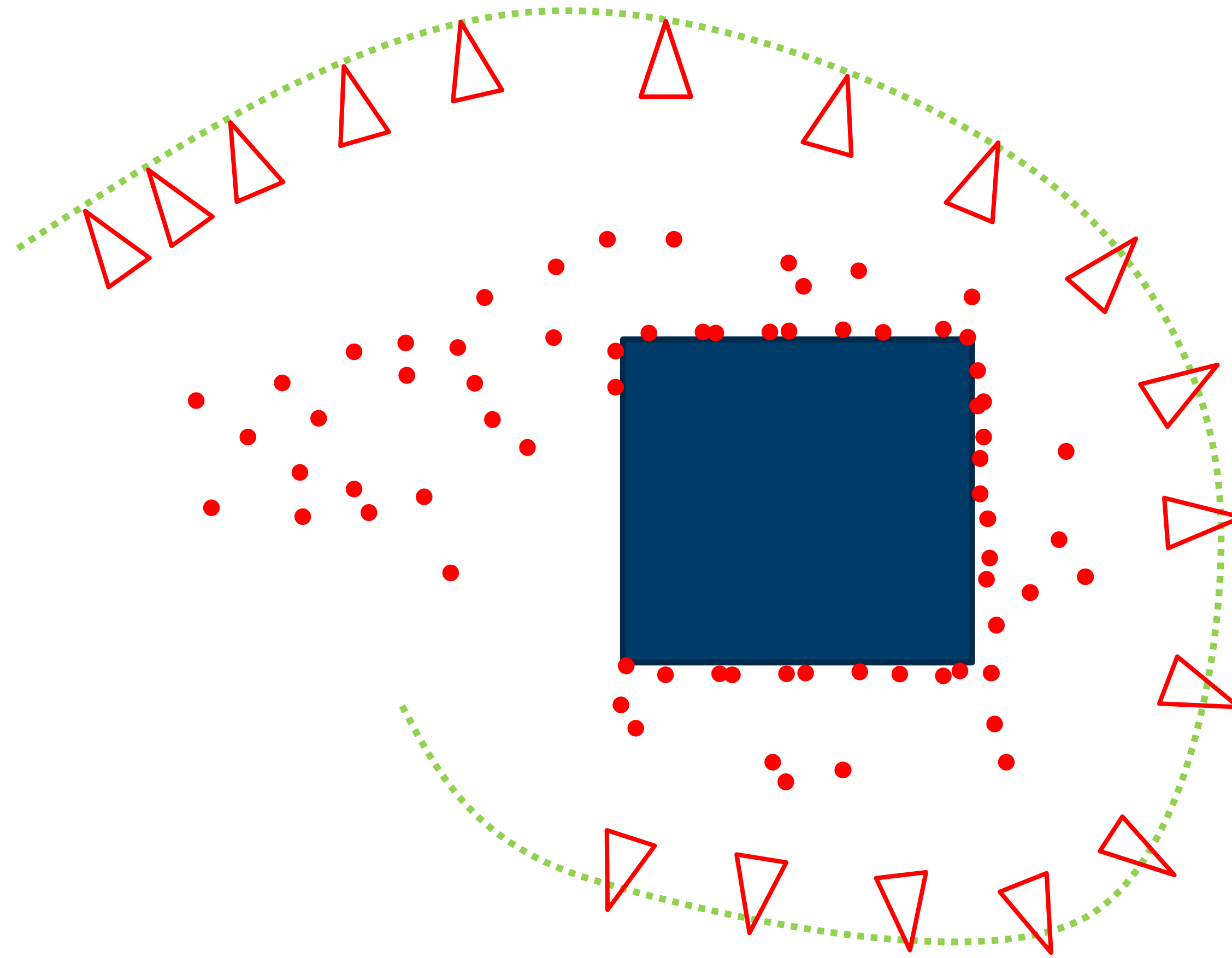
Aspects of a Practical Solution: Sliding Window Mapping



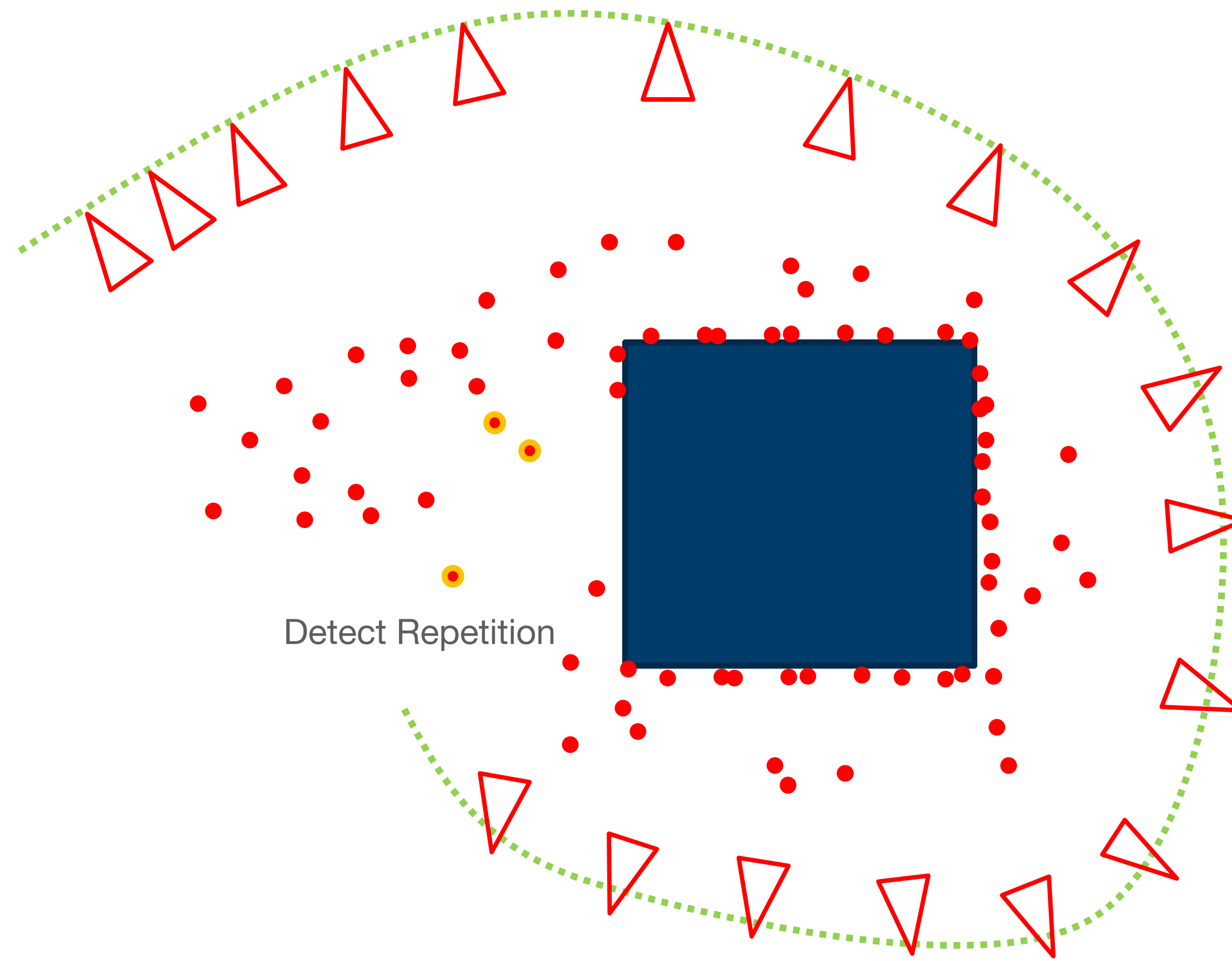
Which can get larger

Aspects of a Practical Solution: Loop Closure

All points in the map so far...

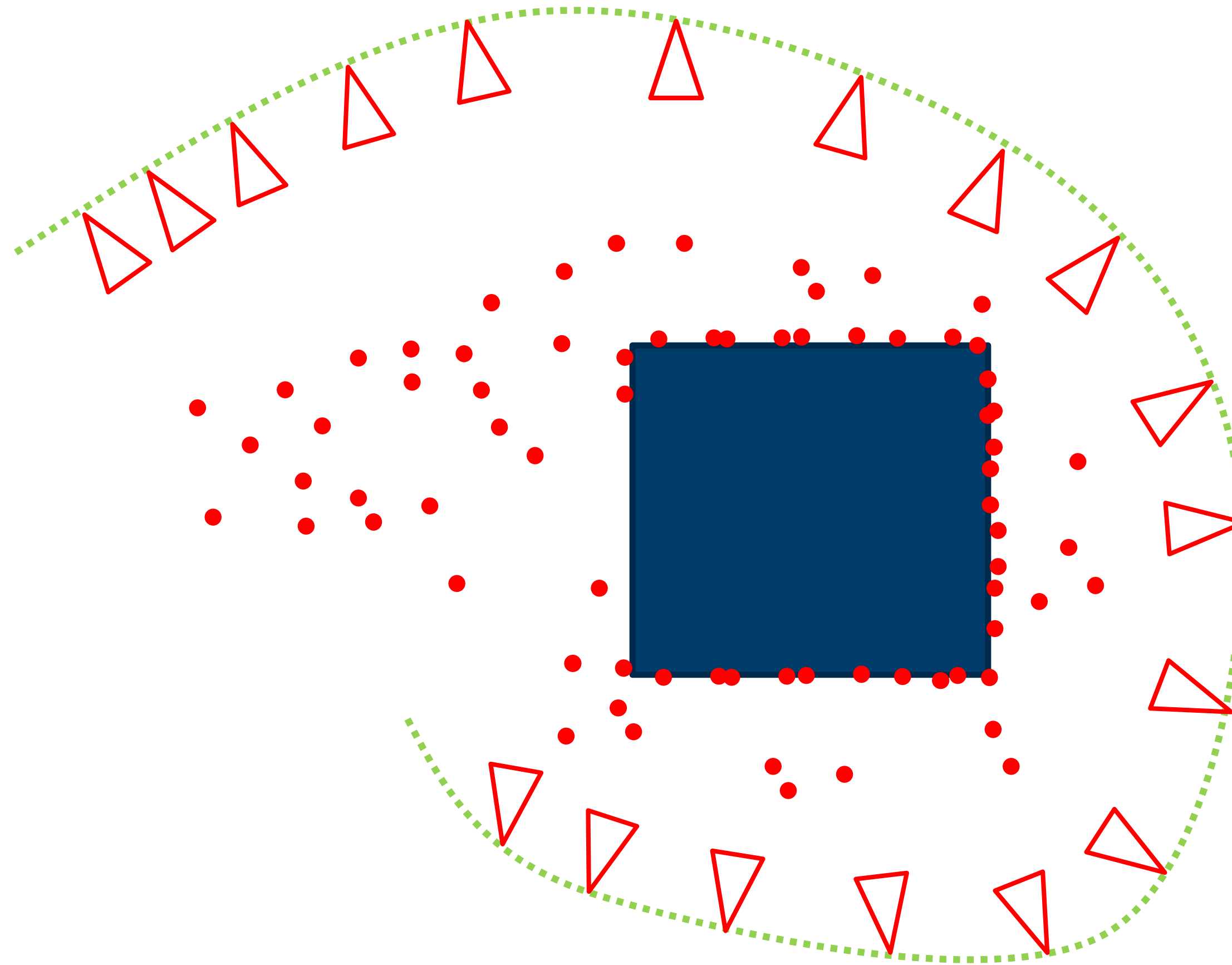


Aspects of a Practical Solution: Loop Closure



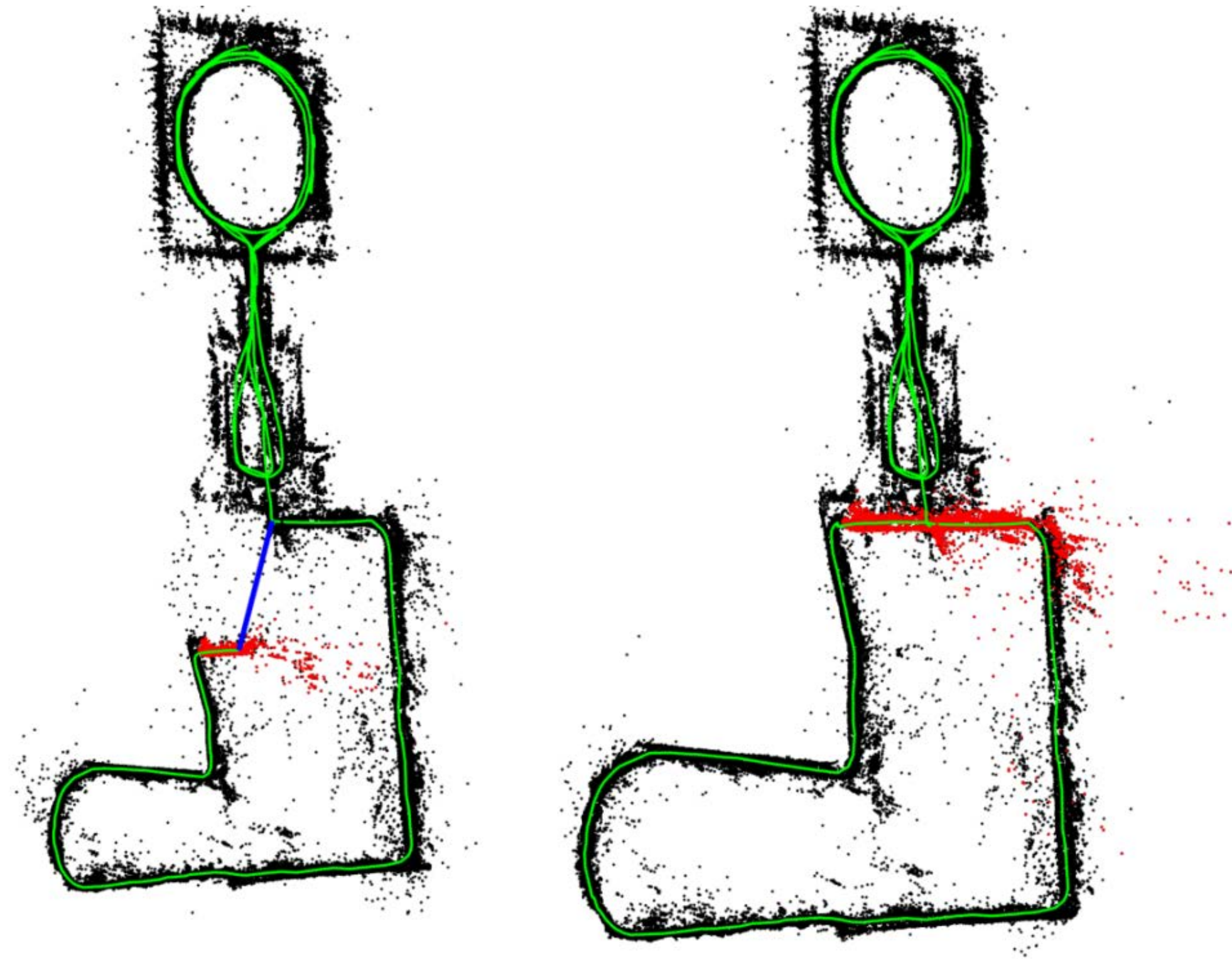
Aspects of a Practical Solution: Loop Closure

Joint refinement of points and poses across keyframes



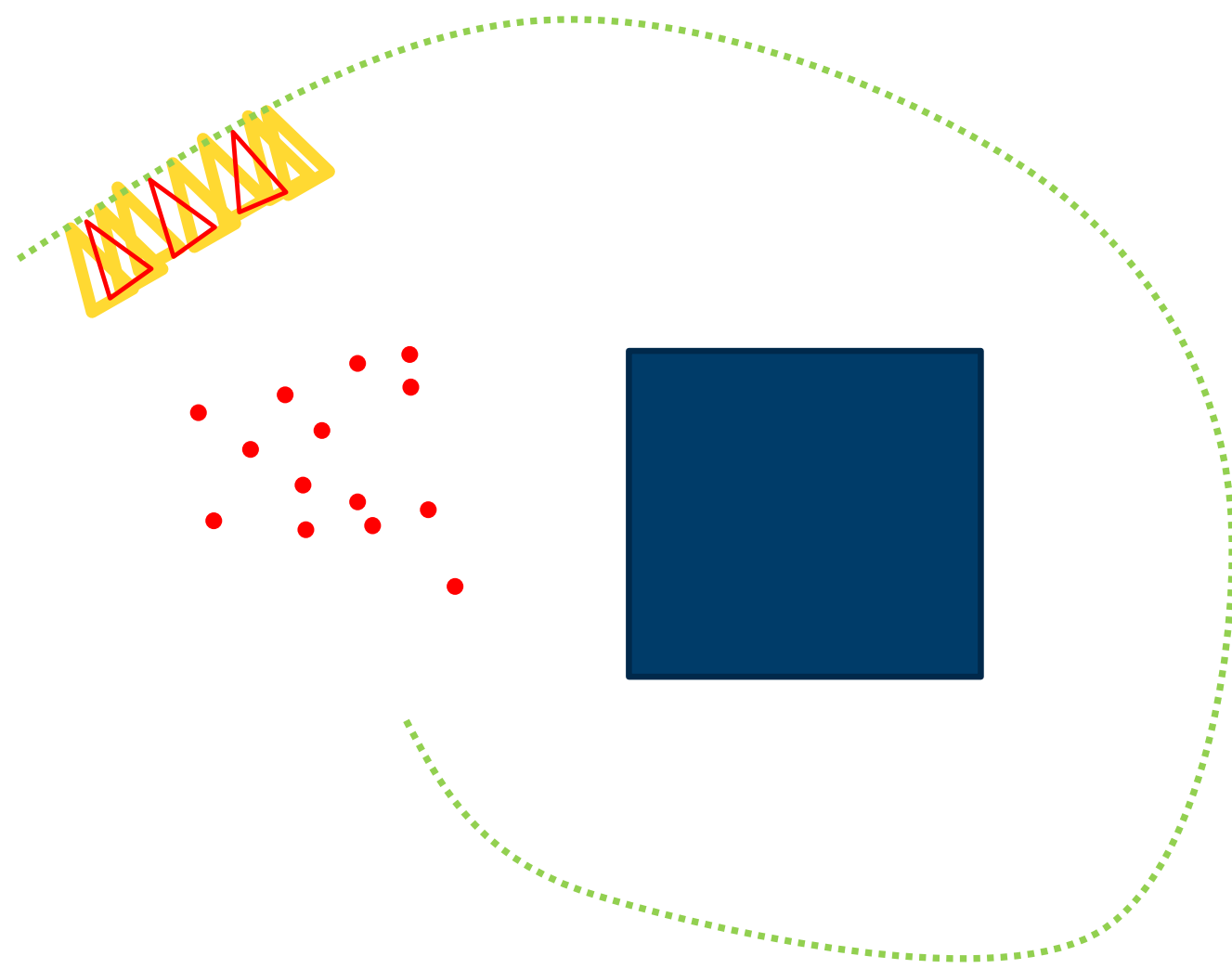
Can be compute intensive — done in a ‘backend’

Aspects of a Practical Solution: Loop Closure

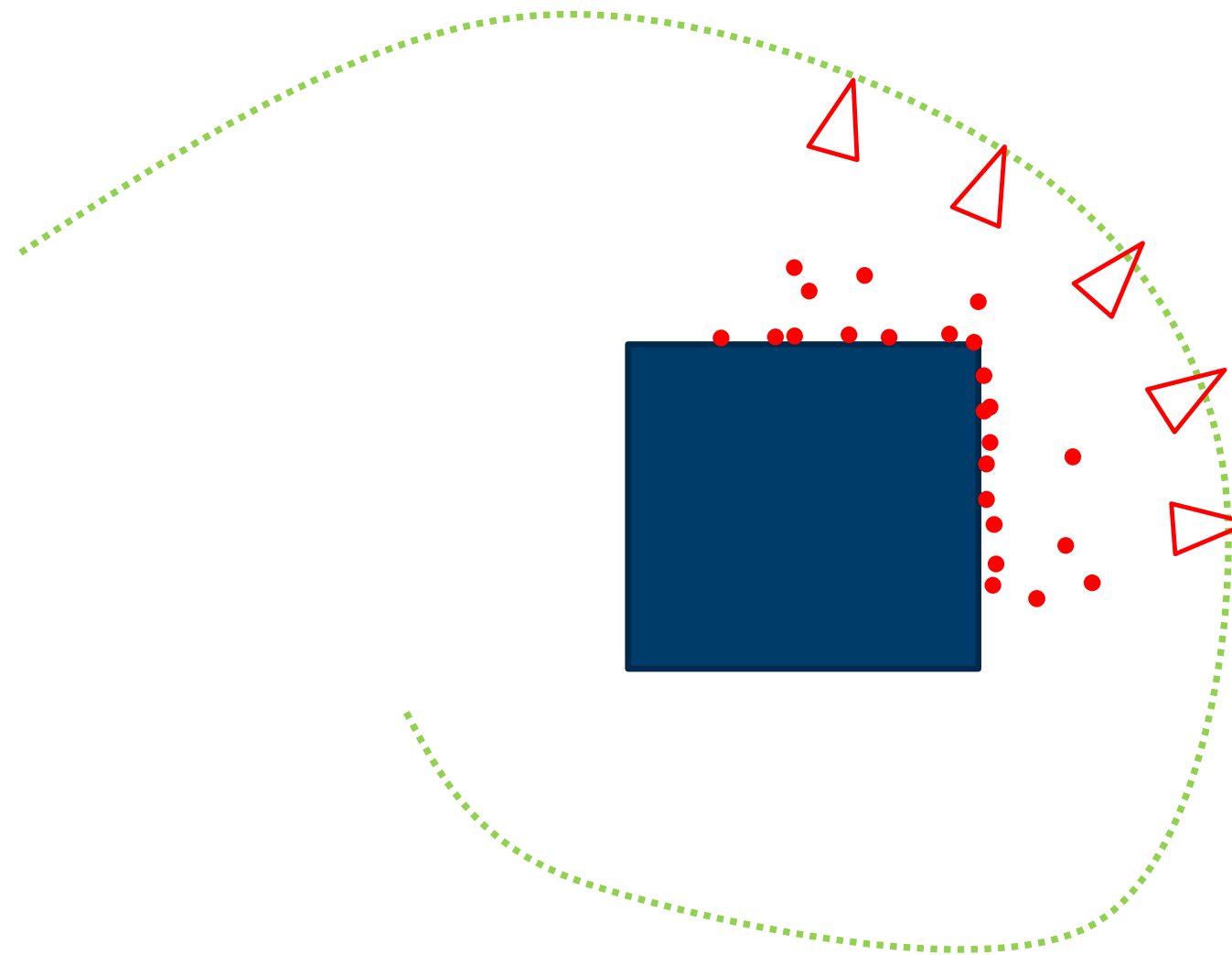


Left: without loop closure, Right: with loop closure

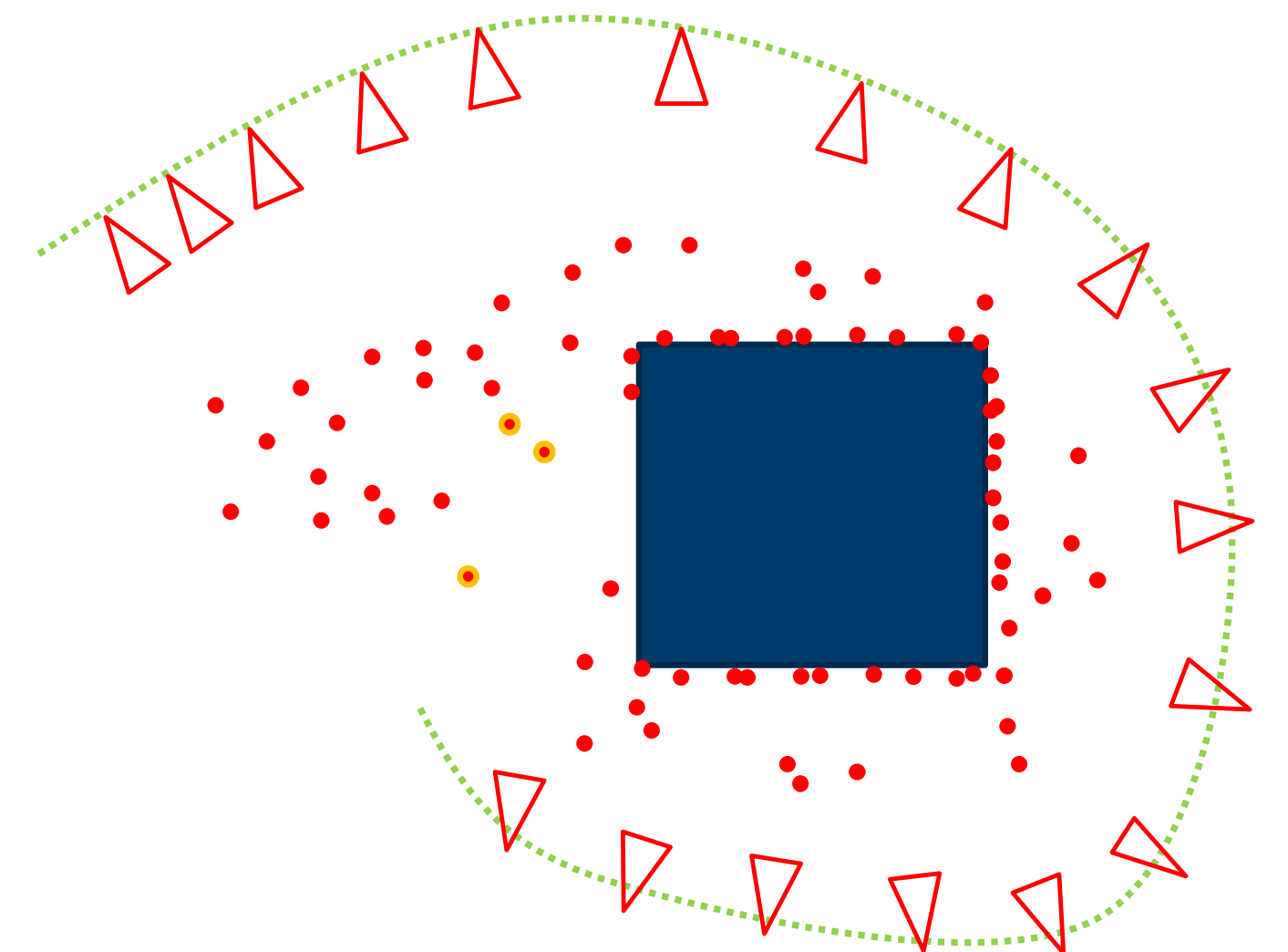
Aspects of a Practical SLAM System



Subsampling KeyFrames



Sliding window mapping



Loop Closure

Let's look at a few example SLAM systems!

ORB-SLAM

Raúl Mur-Artal, J. M. M. Montiel and Juan D. Tardós

{raulmur, josemari, tardos} @unizar.es



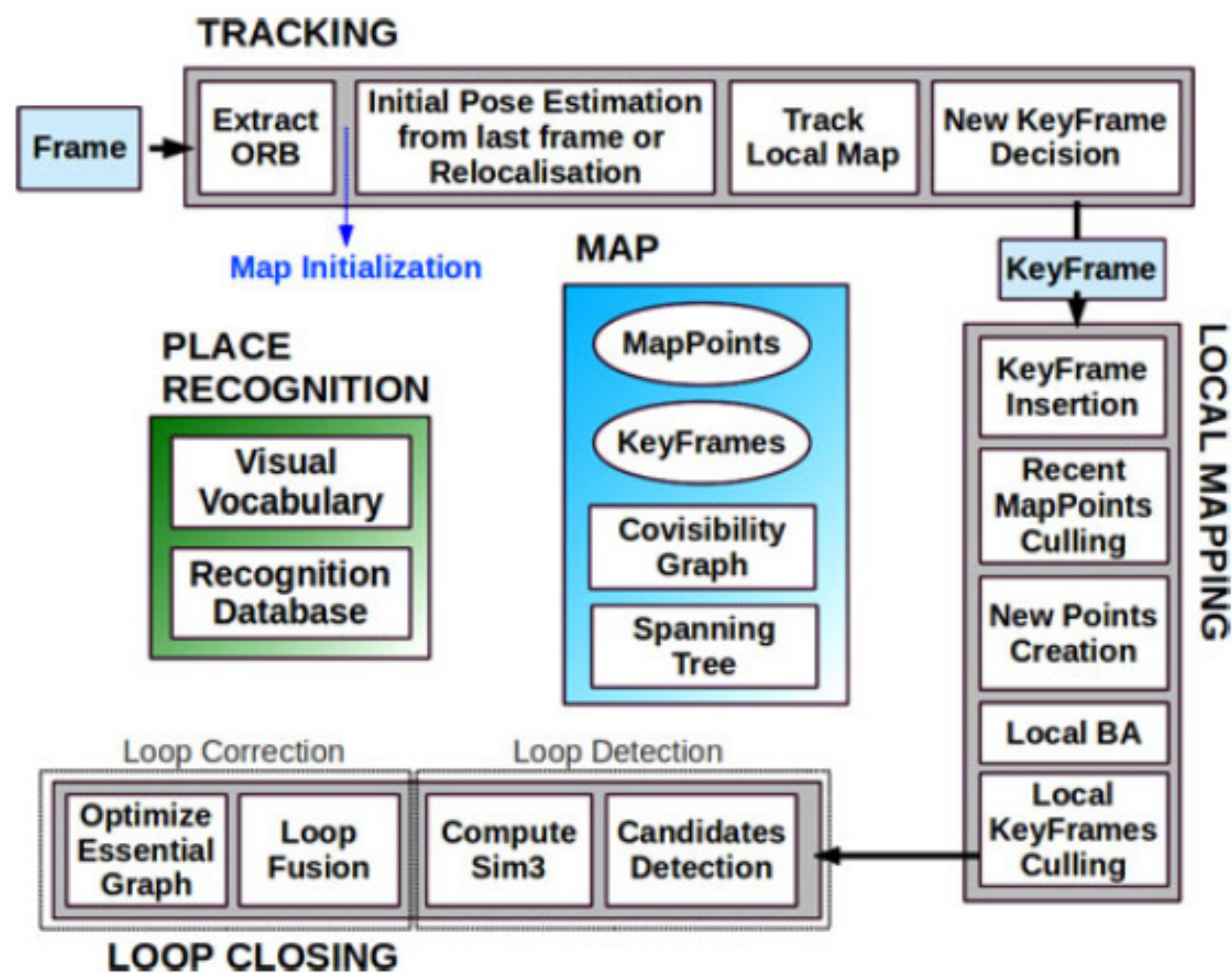
Instituto Universitario de Investigación
en Ingeniería de Aragón
Universidad Zaragoza



Universidad
Zaragoza

System overview

Three parallel threads



Short-term tracking

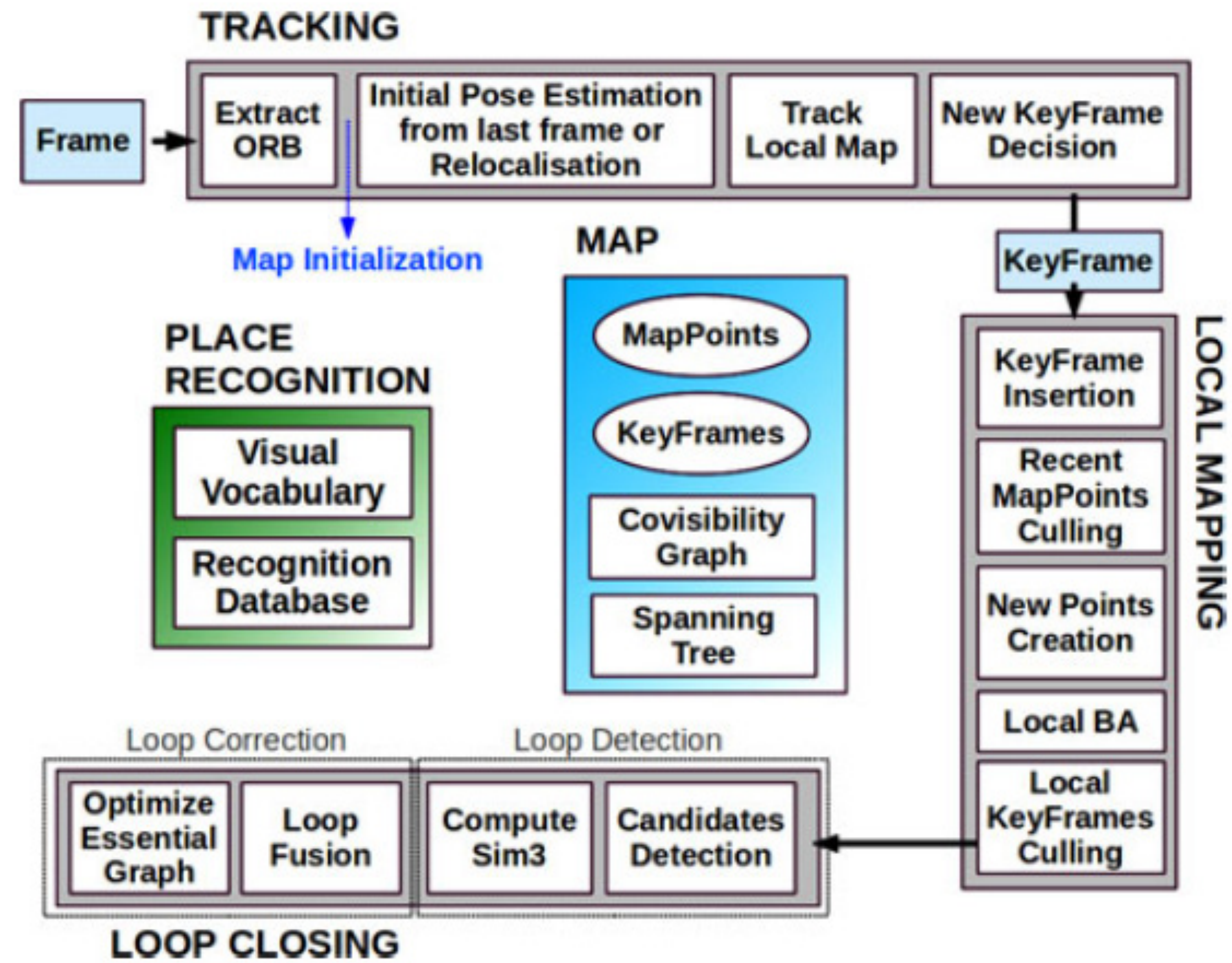
- FAST corners in grid cells at different scale levels with ORB descriptors
- Initial pose estimation:
 - Tracking OK: Guided search with constant velocity model
 - Tracking ~OK: Wider search around positions in previous frame
 - Tracking lost: Global relocalization (long-term tracking)
- Track local map
 - Project local map and search for more correspondences
 - Motion-only bundle adjustment

New keyframe decision

Insert keyframes often to make tracking more robust to rotations

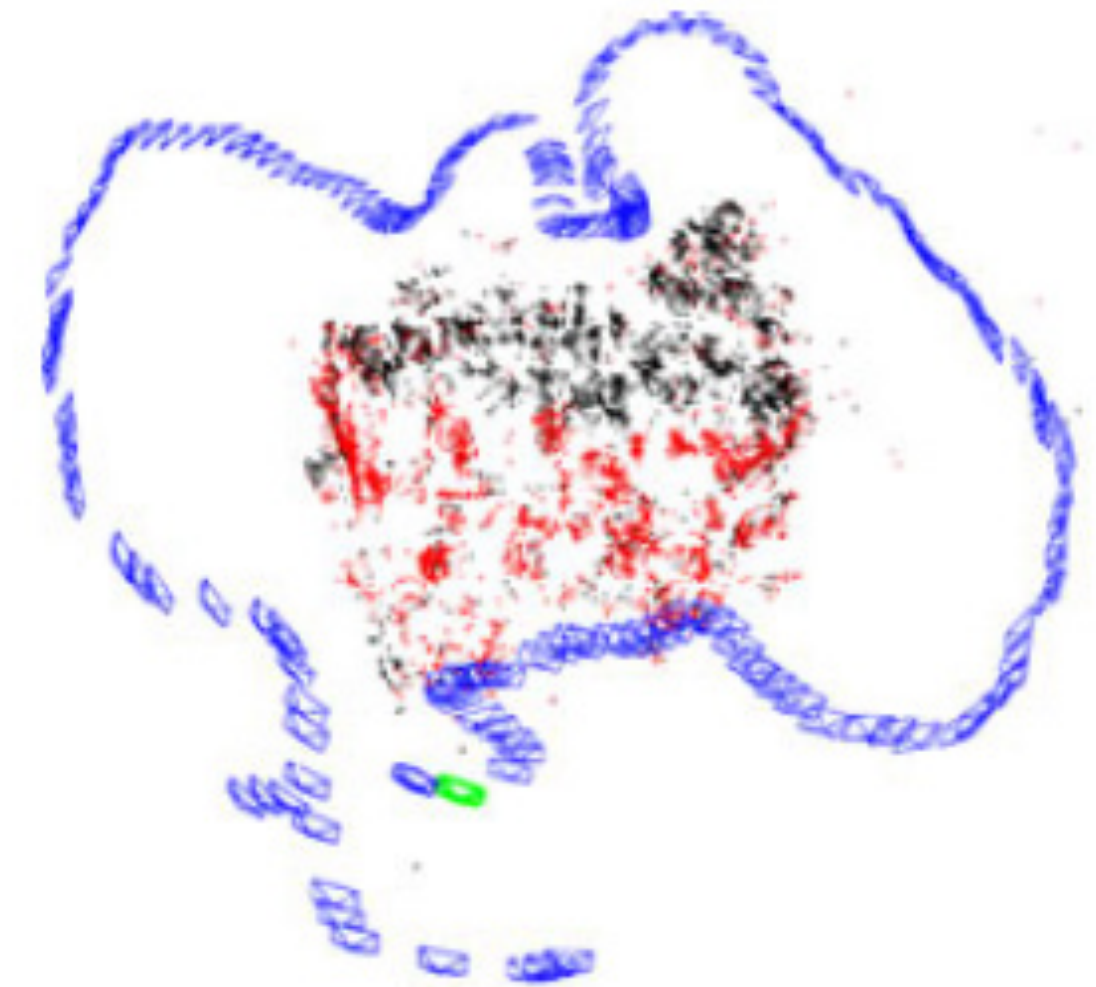
- More than 20 frames since last global relocalization
- Local mapping is idle,
or more than 20 frames since last keyframe insertion
- Current frame tracks at least 50 points
- Current frame tracks less than 90% than that of the reference keyframe

Mapping



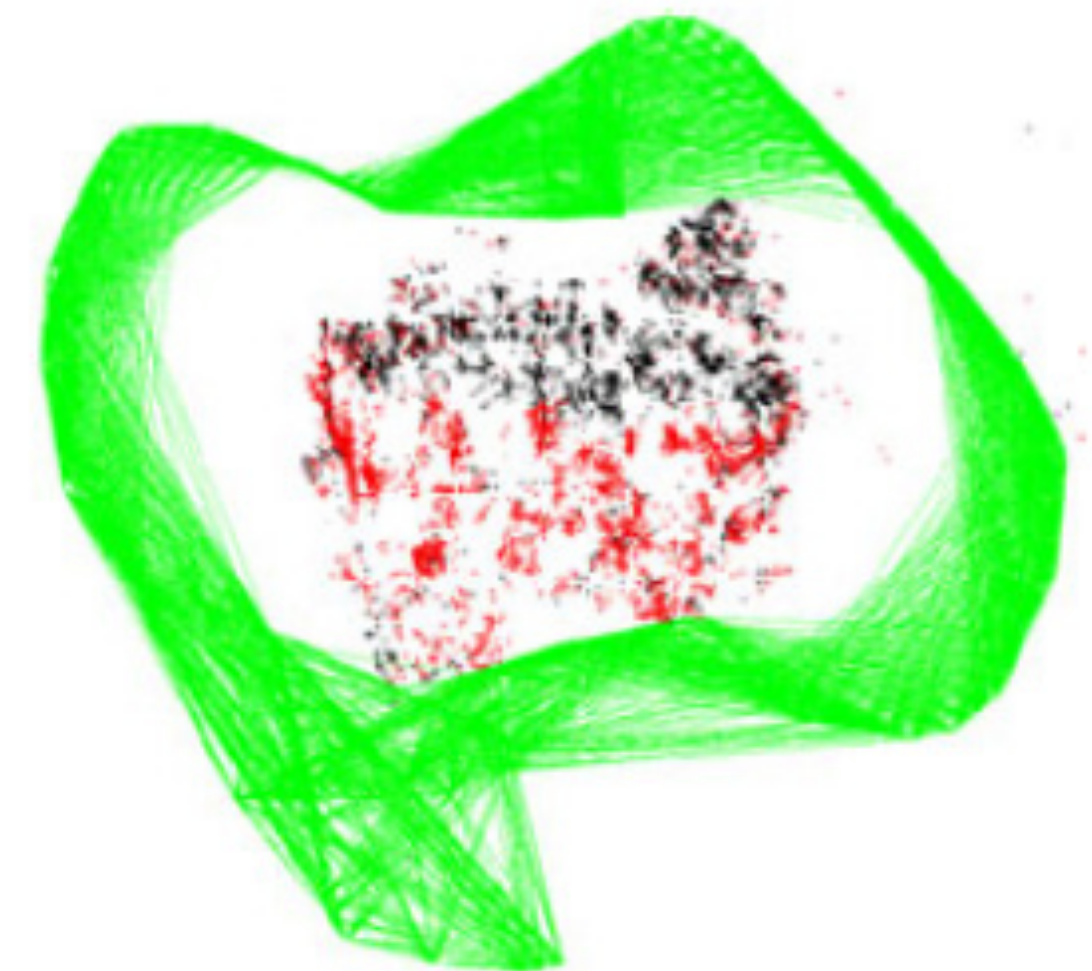
Map

- Keyframes (blue)
- Current frame (green)
- Map points (black)
- Active map points (red)



Map

- Co-visibility graph
 - Nodes: All keyframes
 - Edges: Number of common map points (at least 15)



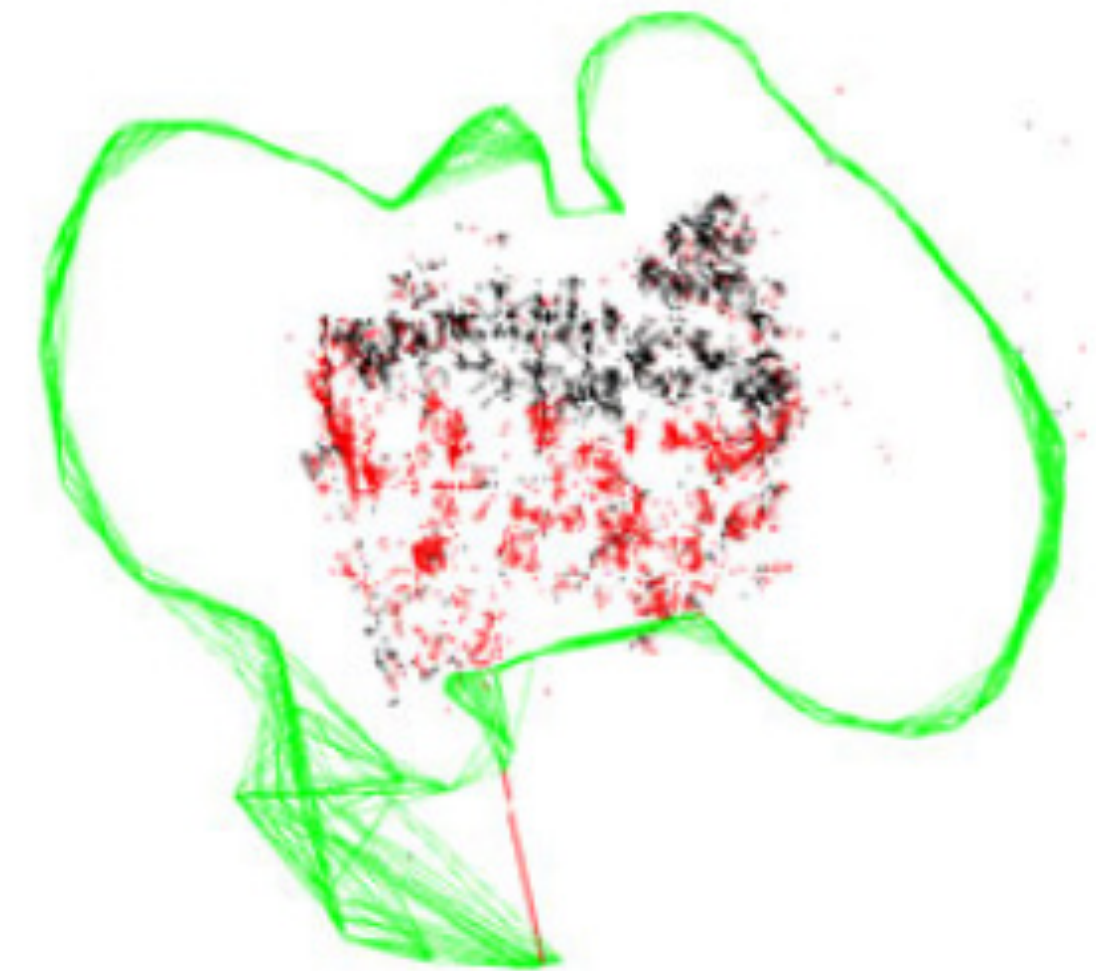
Map

- Spanning tree
 - Connected subgraph of the co-visibility graph with minimal number of strong edges



Map

- Essential graph
 - Spanning tree
 - Subset of edges from the co-visibility graph with high co-visibility (at least 100)
 - Loop closure edges



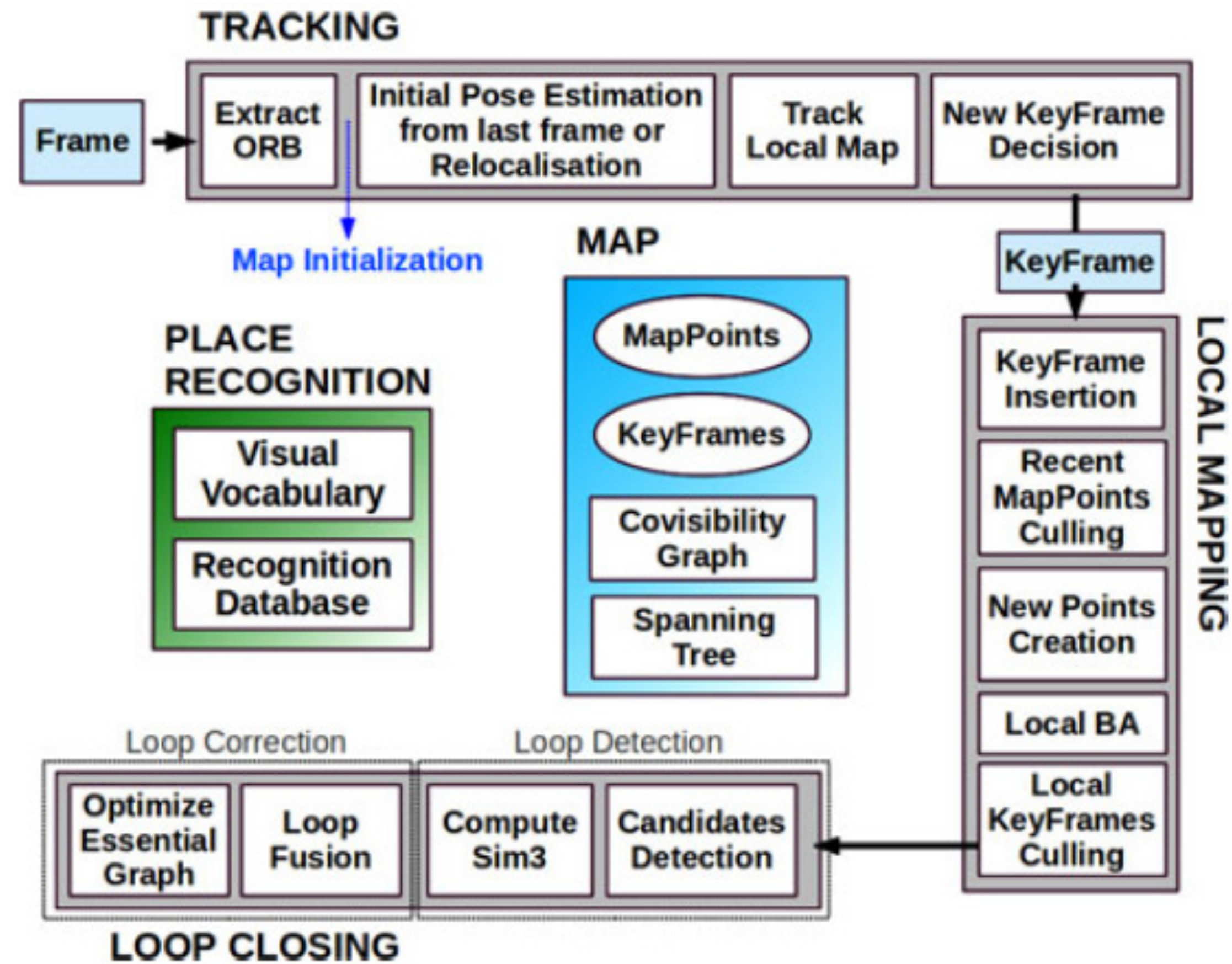
Mapping

- Keyframe insertion
 - Add to co-visibility graph and spanning tree
- Recent map point culling
 - Remove bad points during the first three keyframes after creation
- New map point creation
 - Triangulate ORB from connected keyframes

Mapping

- Local bundle adjustment
 - Optimize the current keyframe, all connected keyframes and all points seen
 - All other keyframes remain fixed
- Local keyframe culling
 - Detect and delete redundant keyframes

Long-term tracking



Long-term tracking

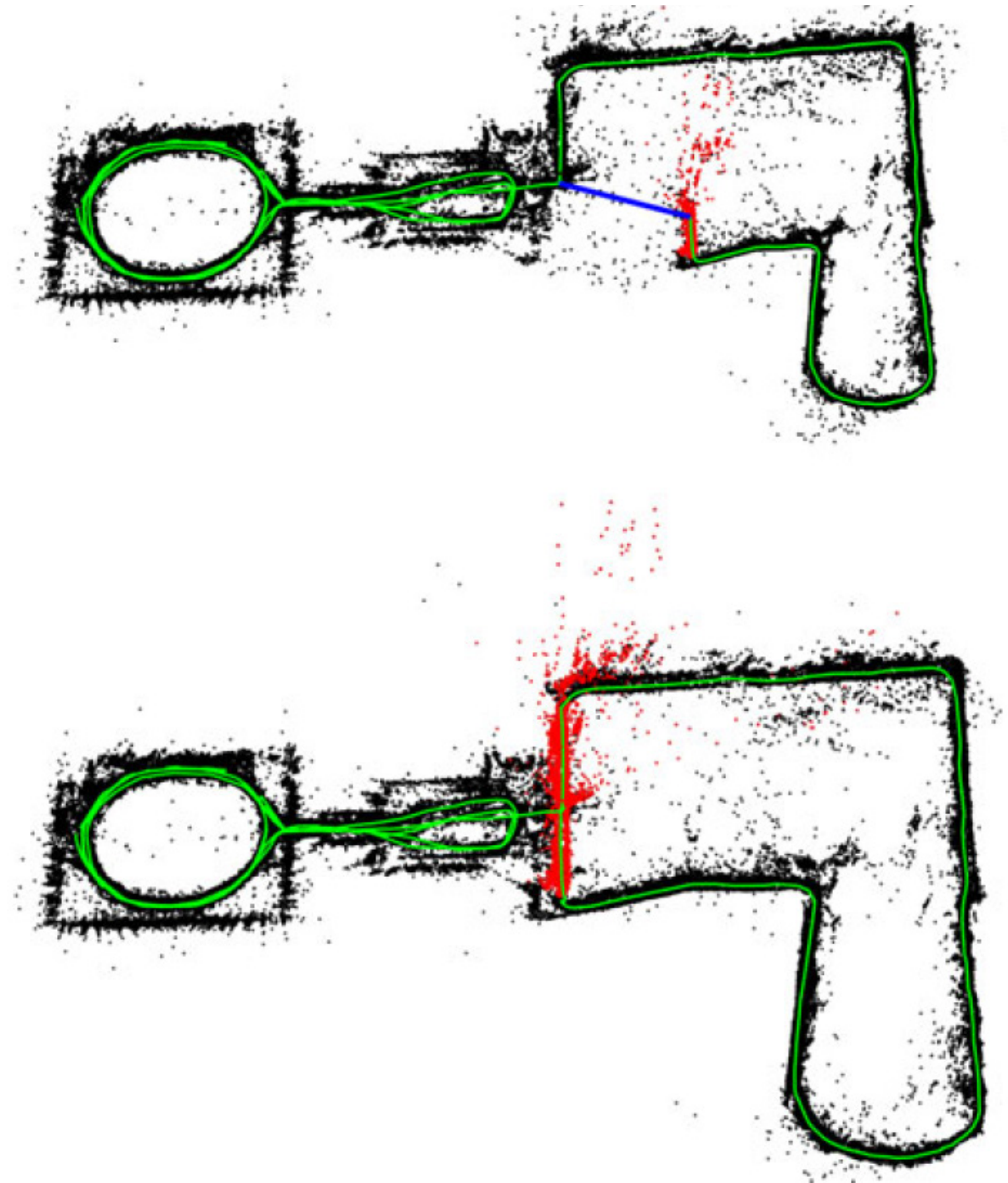
- Long-term tracking is performed on the last keyframe in its own thread
- Loop closure candidates:
 - Compute BoW threshold based on the lowest score for the neighboring keyframes
 - Query recognition database (DBoW2) for keyframes with score higher than the threshold
 - Keep those that are not directly connected, and where we have at least three connected candidates

Long-term tracking

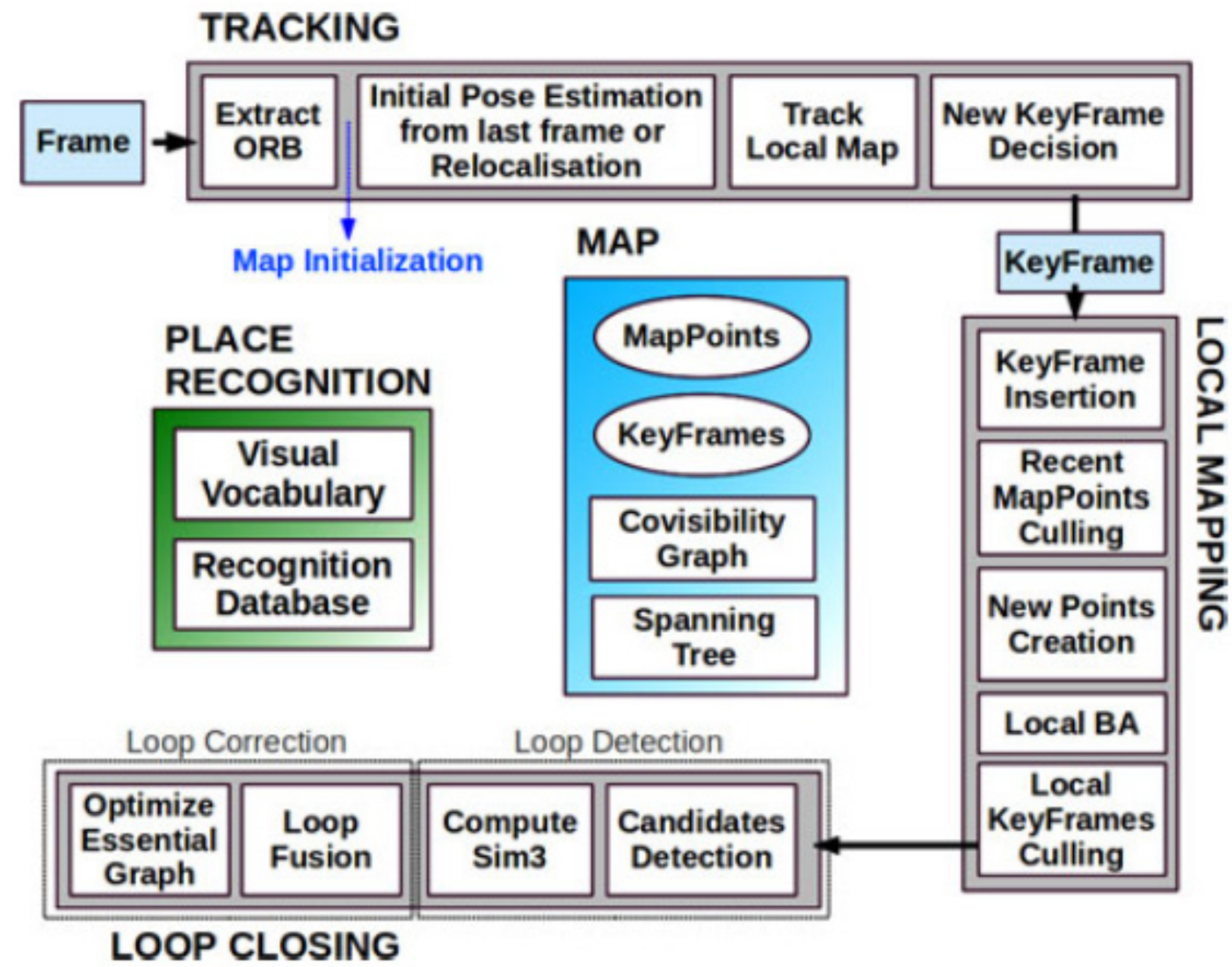
- Scale-drift aware loop closure alignment
 - Compute an initial similarity transform between the current keyframe and the loop keyframe from 3D-to-3D correspondences
 - Search for more correspondences
 - Optimize again
 - Geometric validation: Accept loop if enough inliers

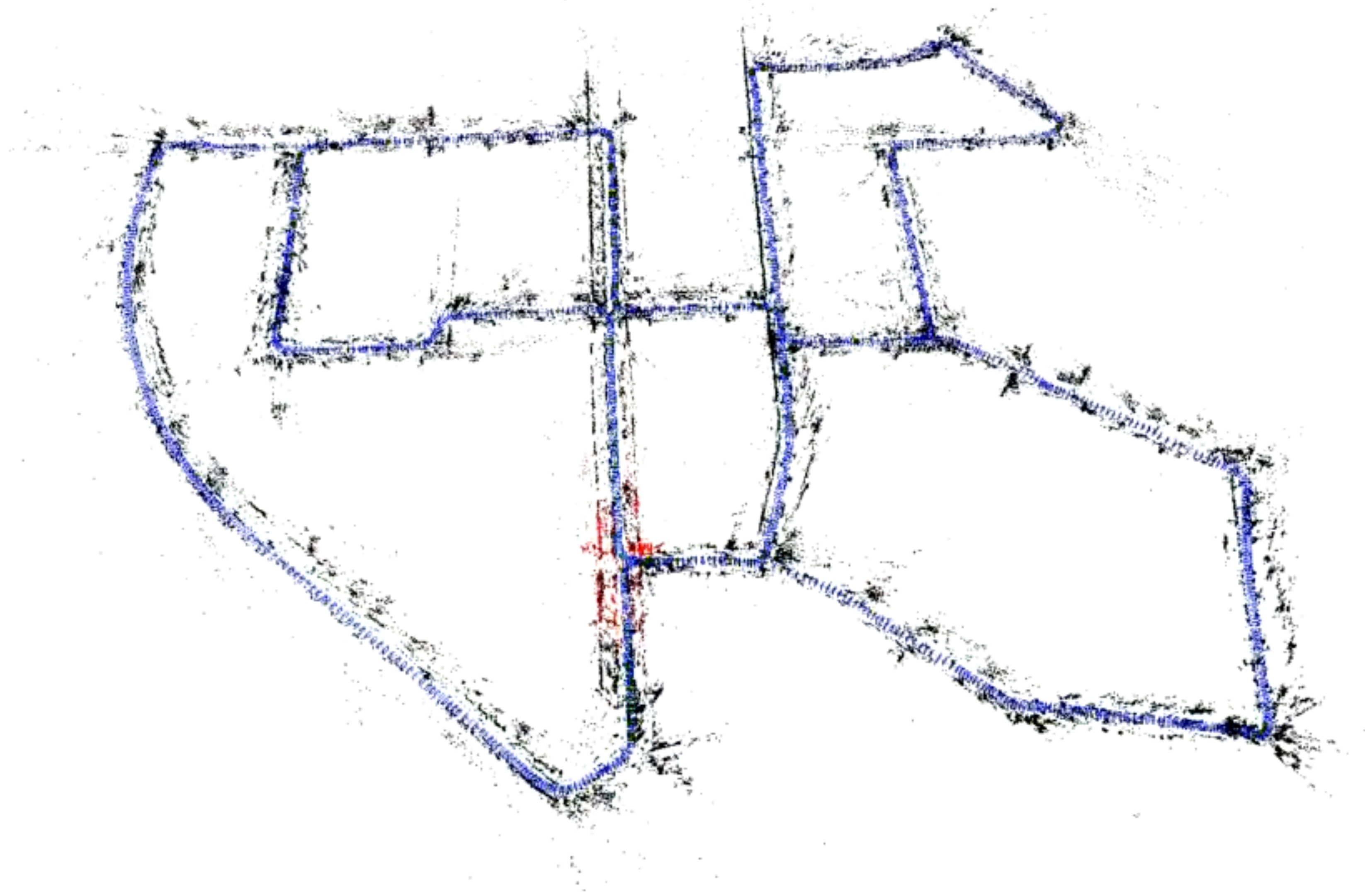
Loop correction

- Loop fusion
 - Fuse map points
 - Insert new edges in the co-visibility graph
- Essential graph optimization
 - Distribute the loop closing error along a pose graph over $\mathbf{sim}(3)$
 - Transform each map point according to the correction of one of the keyframes that observes it



System overview





ORB-SLAM2: an Open-Source SLAM System for Monocular, Stereo and RGB-D Cameras

Raúl Mur-Artal and Juan D. Tardós

2017

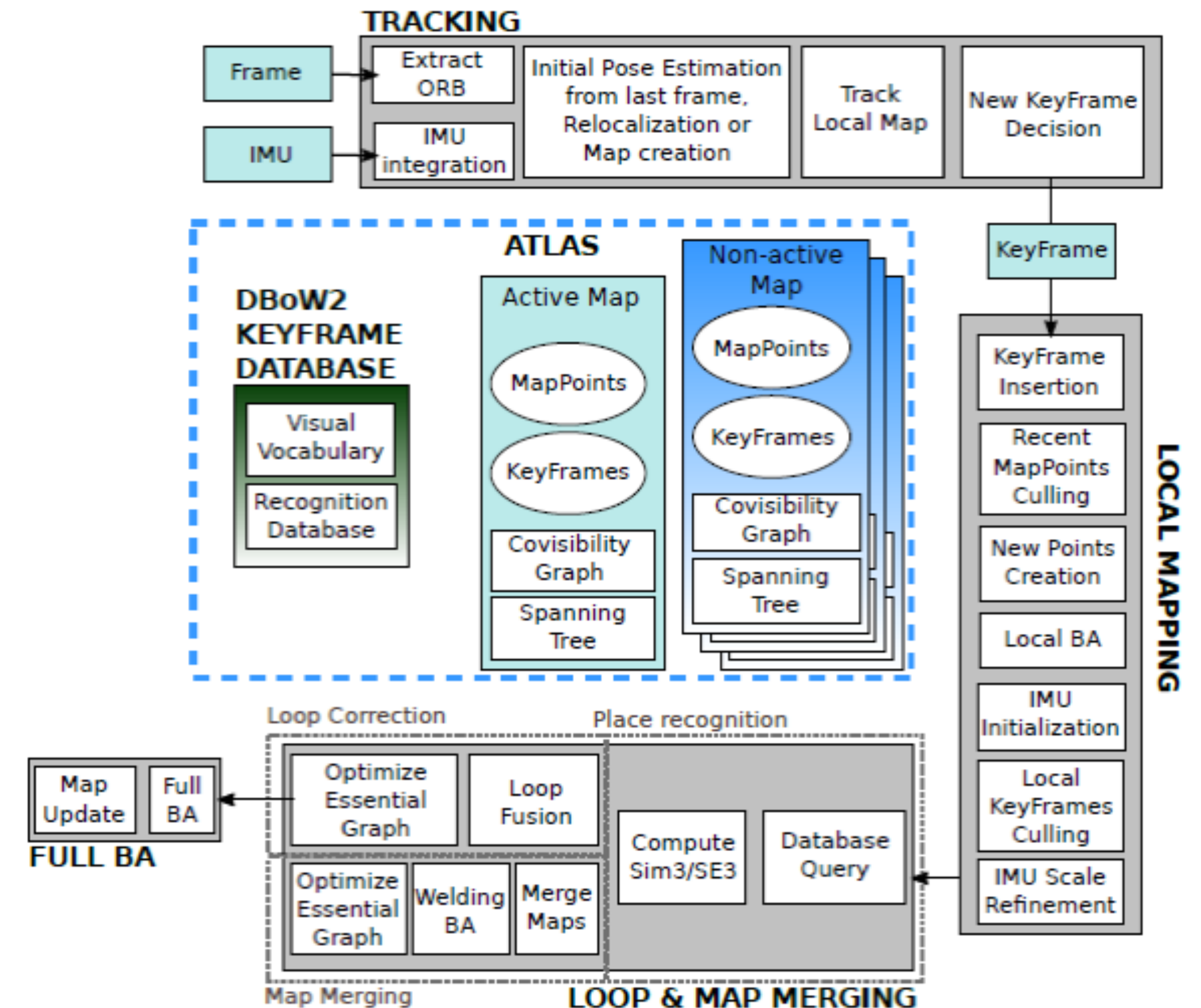
- Extension to stereo and RGBD
- Localization mode that localizes to map (without updating map) and uses frame-to-frame tracking when off map

ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM

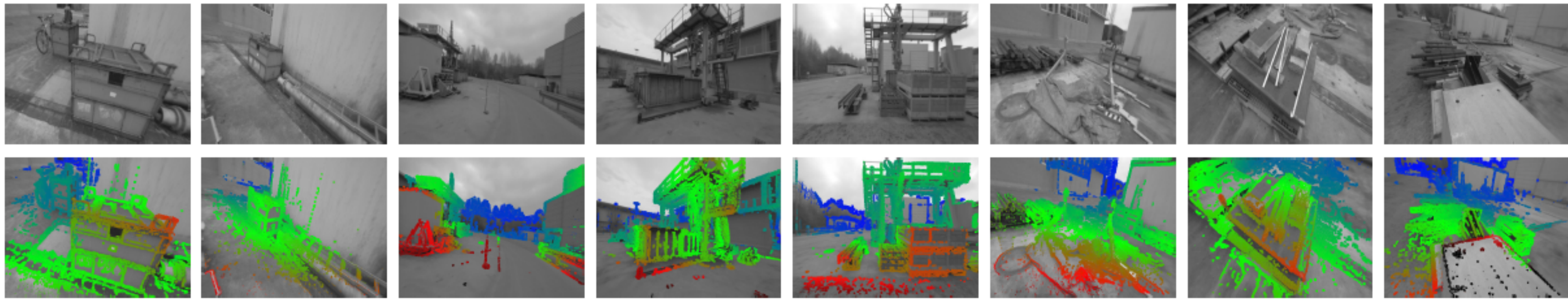
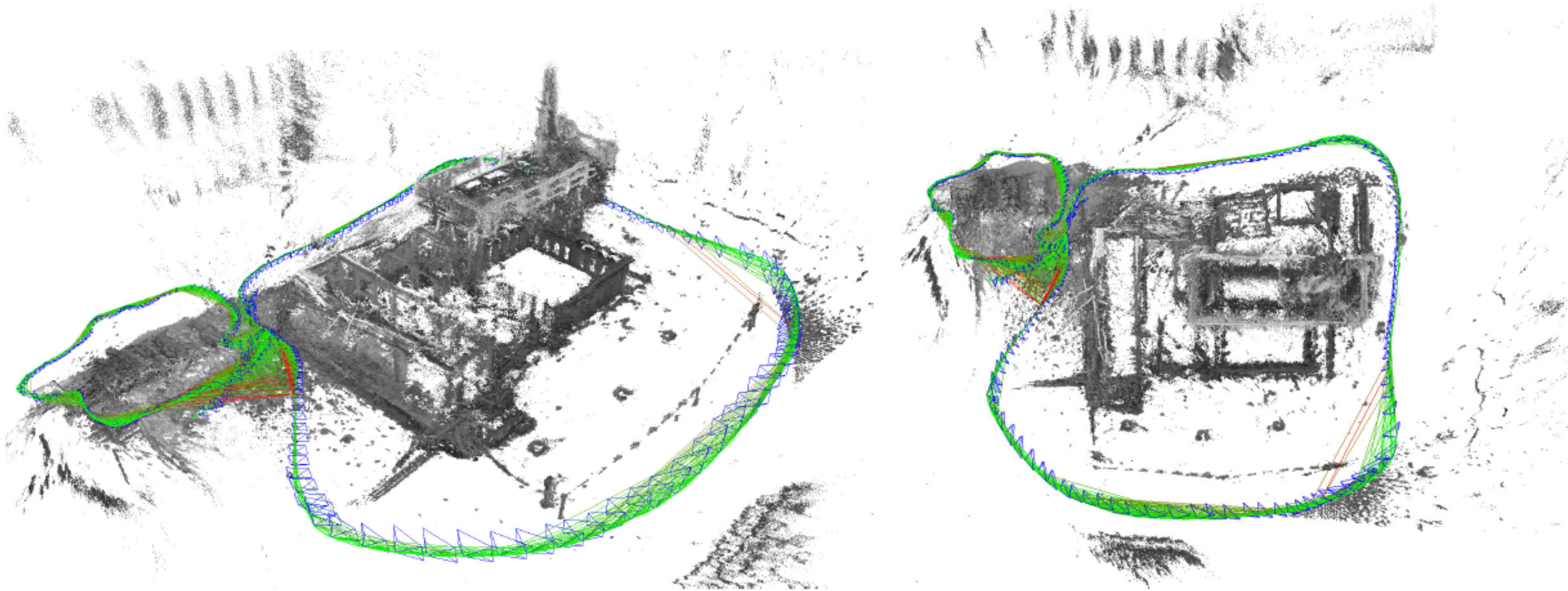
Carlos Campos*, Richard Elvira*, Juan J. Gómez Rodríguez, José M.M. Montiel and Juan D. Tardós

2021

- Incorporates IMU (inertial measurement unit)
- Maintains several maps so can build over multiple sessions or start new map if tracking lost and merge later

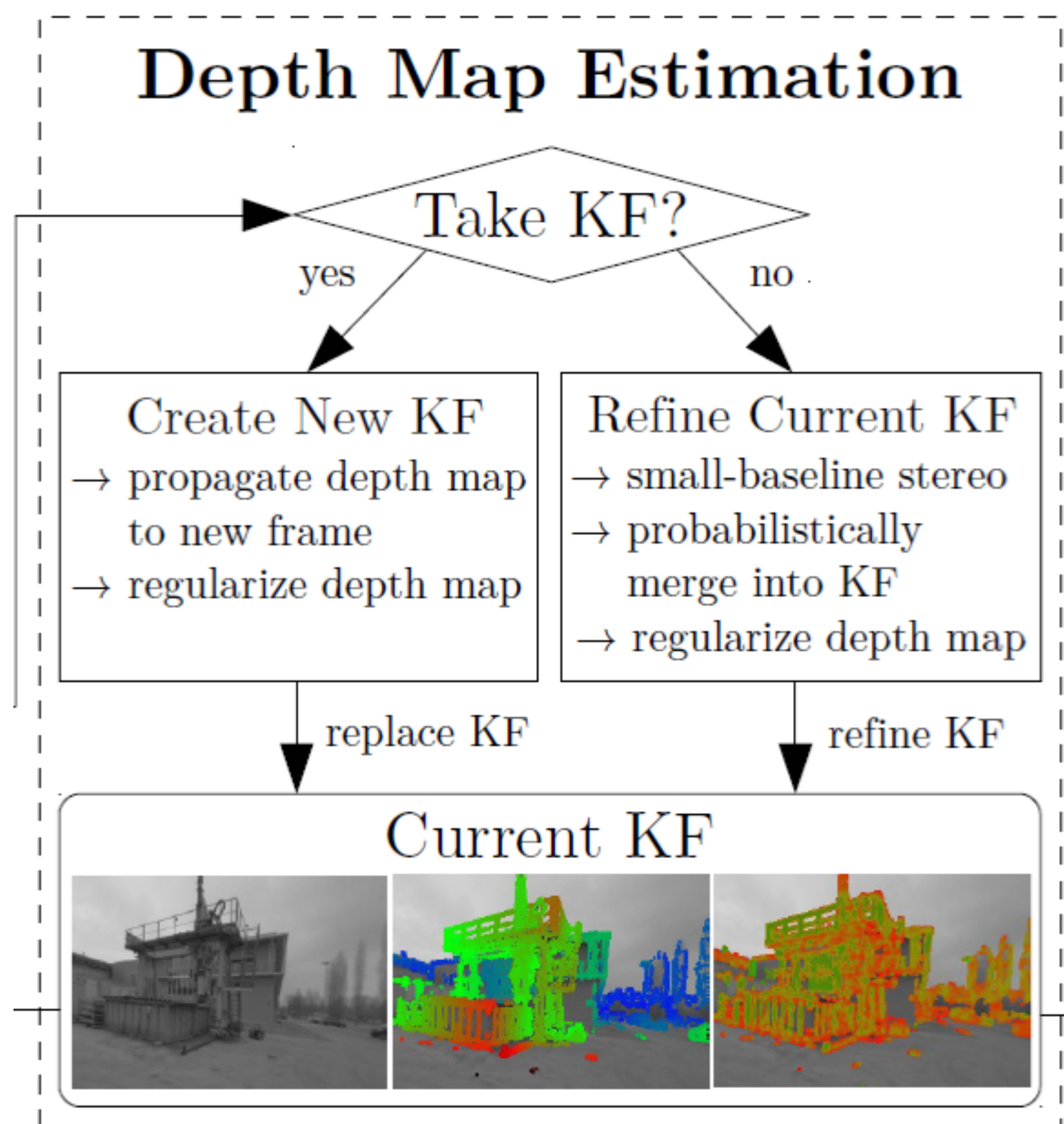


LSD-SLAM [Engel, Schops, Cremers, ECCV 2014]



LSD-SLAM: Overview (Large-Scale Direct SLAM)

- “Direct” visual odometry
 - Matching/alignment uses pixel correlation instead of features
- Dense matches in textured/edge regions
- Alignment between keyframes estimates similarity transform to account for scale drift
- Depth and uncertainty estimated per pixel



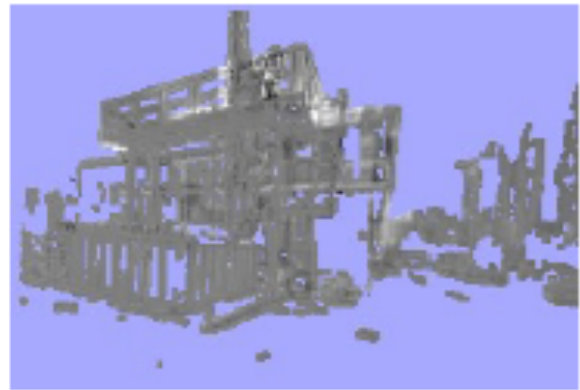
Tracking

New Image
(640 x 480 at 30Hz)



Track on Current KF:

→ estimate SE(3) transformation



$$\min_{\xi \in \mathfrak{se}(3)} \sum_{\mathbf{p}} \left\| \frac{r_p^2(\mathbf{p}, \xi)}{\sigma_{r_p}^2(\mathbf{p}, \xi)} \right\|_{\delta}$$

tracking reference

Depth Map Estimation

Take KF?

yes

no

Create New KF

→ propagate depth map
to new frame
→ regularize depth map

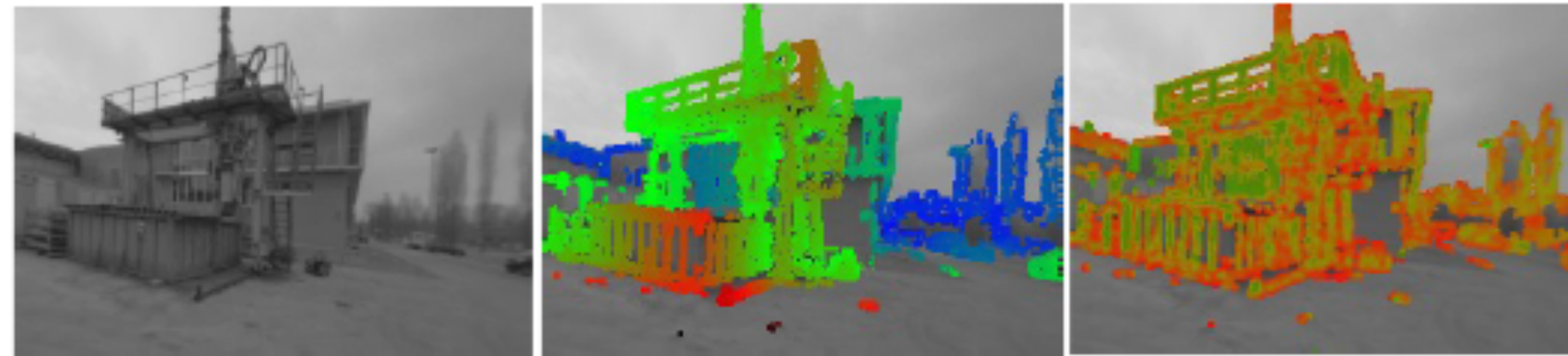
Refine Current KF

→ small-baseline stereo
→ probabilistically
merge into KF
→ regularize depth map

replace KF

refine KF

Current KF



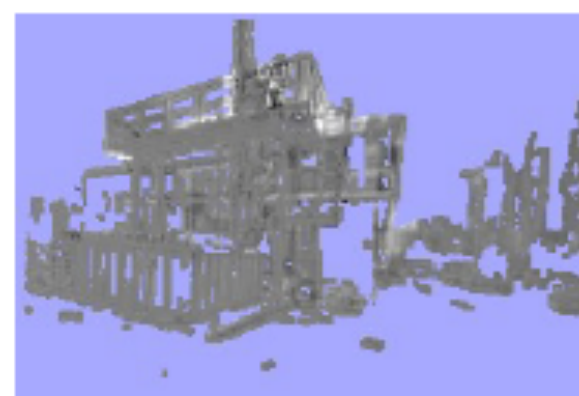
Tracking

New Image
(640 x 480 at 30Hz)



Track on Current KF:

→ estimate SE(3) transformation



$$\min_{\xi \in \mathfrak{se}(3)} \sum_{\mathbf{p}} \left\| \frac{r_p^2(\mathbf{p}, \xi)}{\sigma_{r_p}^2(\mathbf{p}, \xi)} \right\|_{\delta}$$

tracking reference

Depth Map Estimation

Take KF?

yes

no

Create New KF

→ propagate depth map
to new frame
→ regularize depth map

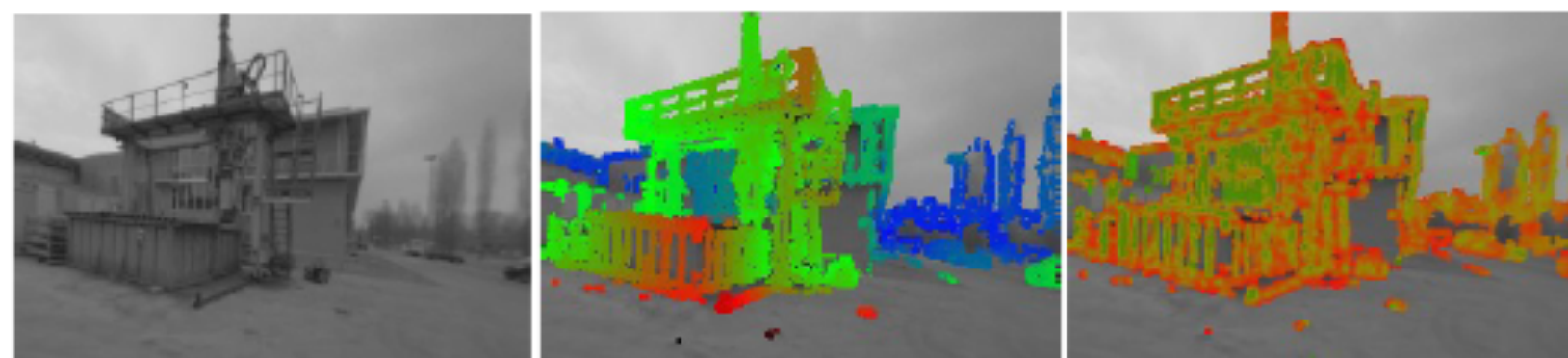
Refine Current KF

→ small-baseline stereo
→ probabilistically
merge into KF
→ regularize depth map

replace KF

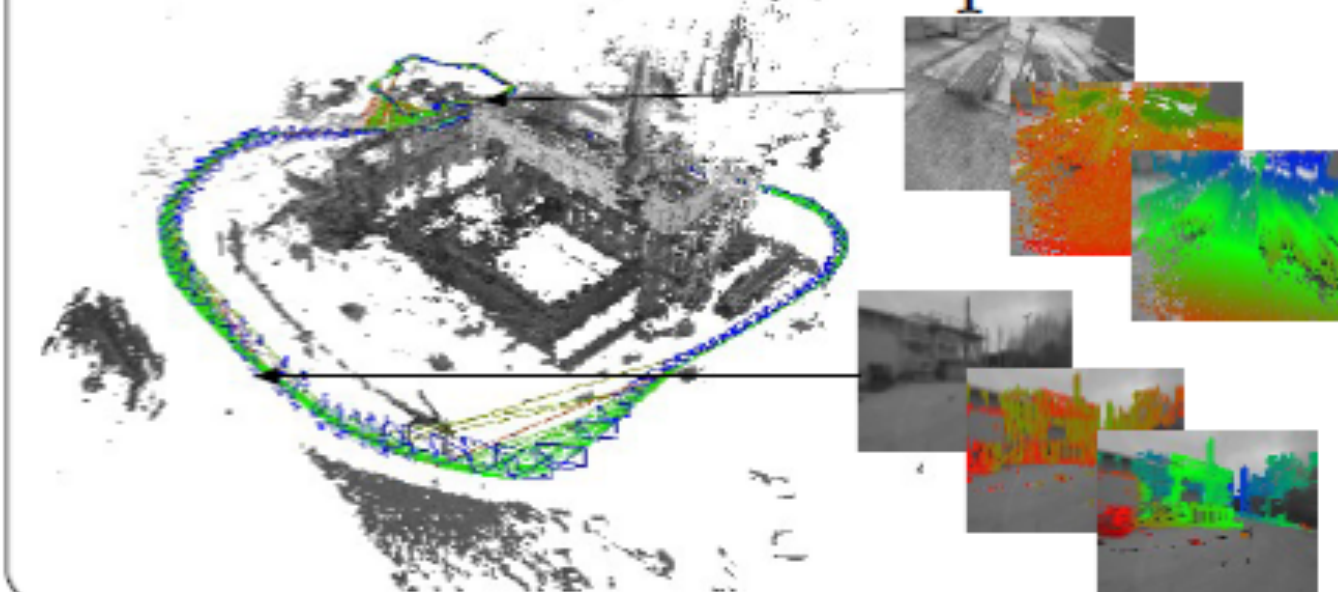
refine KF

Current KF



Map Optimization

Current Map

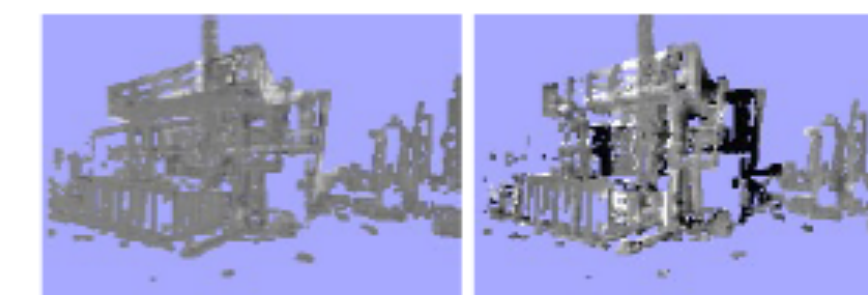


add to map

Add KF to Map

→ find closest keyframes
→ estimate Sim(3) edges

$$\min_{\xi \in \mathfrak{sim}(3)} \sum_{\mathbf{p}} \left\| \frac{r_p^2(\mathbf{p}, \xi)}{\sigma_{r_p}^2(\mathbf{p}, \xi)} + \frac{r_d^2(\mathbf{p}, \xi)}{\sigma_{r_d}^2(\mathbf{p}, \xi)} \right\|_{\delta}$$



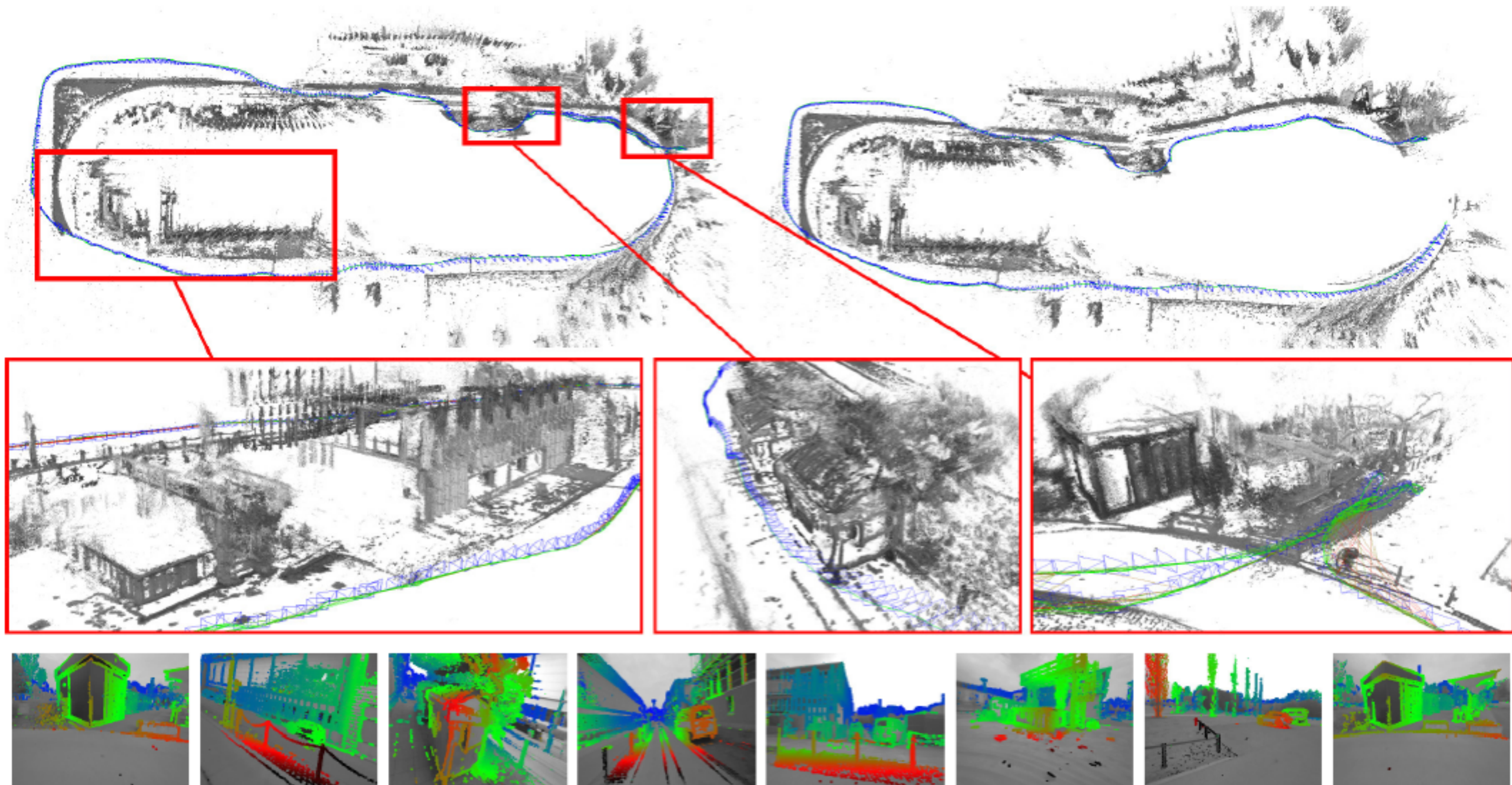


Fig. 7: Loop closure for a long and challenging outdoor trajectory (after the loop closure on the left, before on the right). Also shown are three selected close-ups of the generated pointcloud, and semi-dense depth maps for selected keyframes.

DROID-SLAM

Deep Visual SLAM for Monocular, Stereo
and RGB-D Cameras

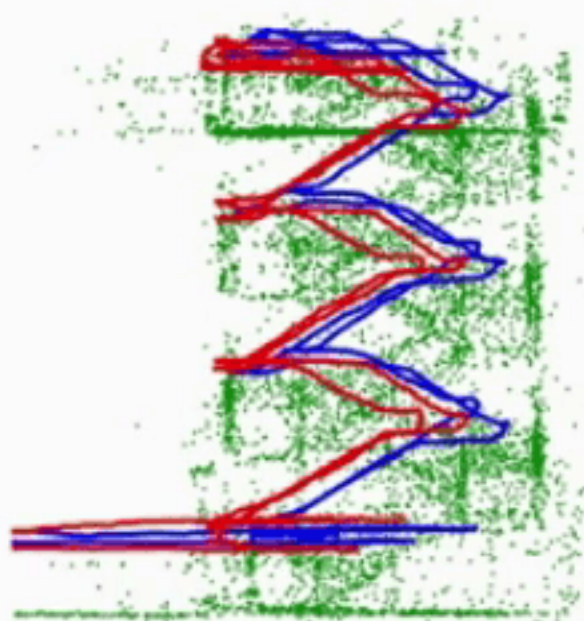
Zachary Teed and Jia Deng
Princeton University



Classical Visual SLAM

Indirect SLAM

- Detects and matches keypoints
- Minimizes *geometric* error between
 - Observed 2D keypoints
 - Projected 3D points



SVO [Forster et al. 2014]

ORB-SLAM [Mur-Artal et al., 2015]

Direct SLAM

- Directly optimize using sensor measurements
- Minimizes *photometric* error



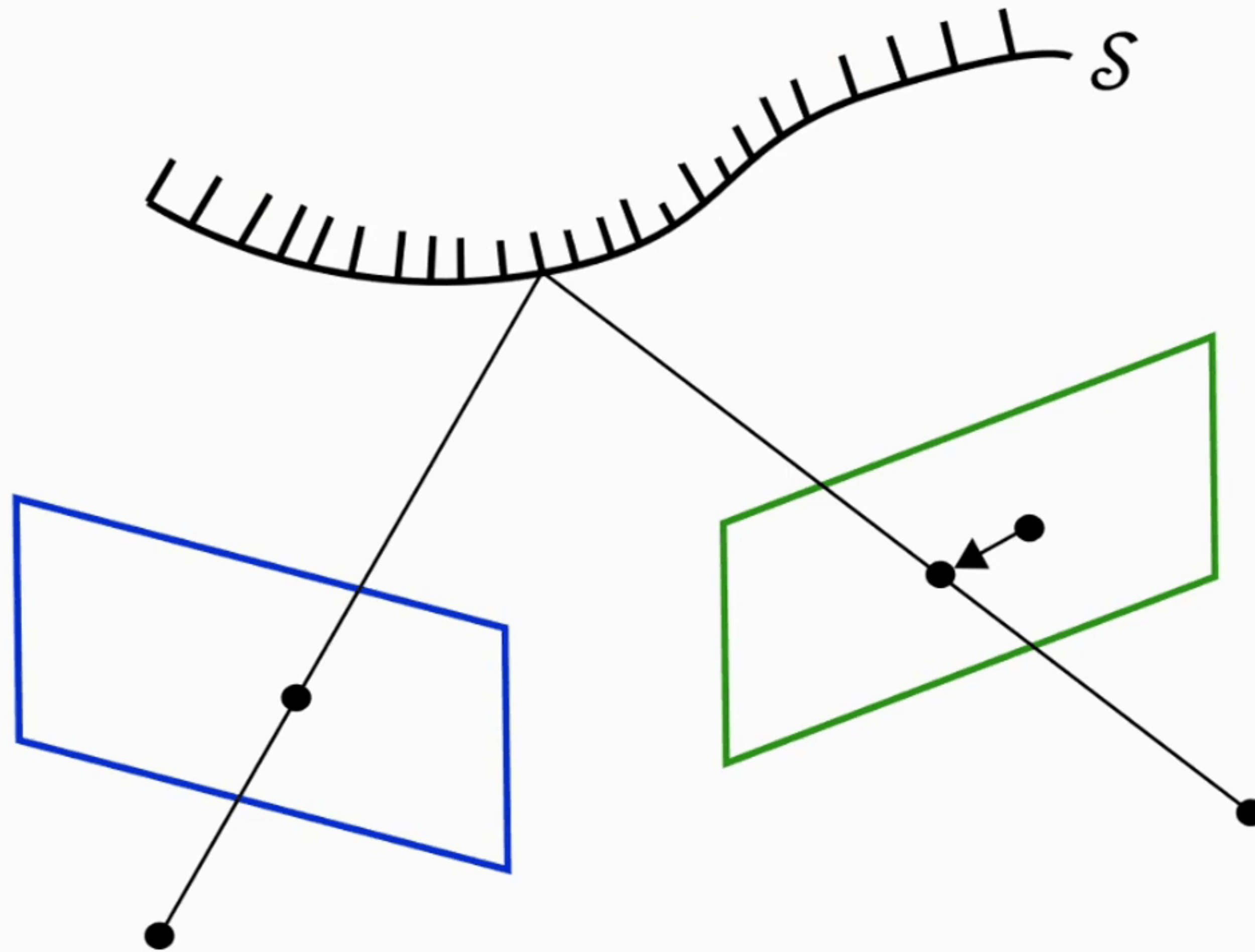
LSD-SLAM [Engel et al., 2014]

DSO [Engel et al. 2016]

BAD-SLAM [Schops et al., 2020]

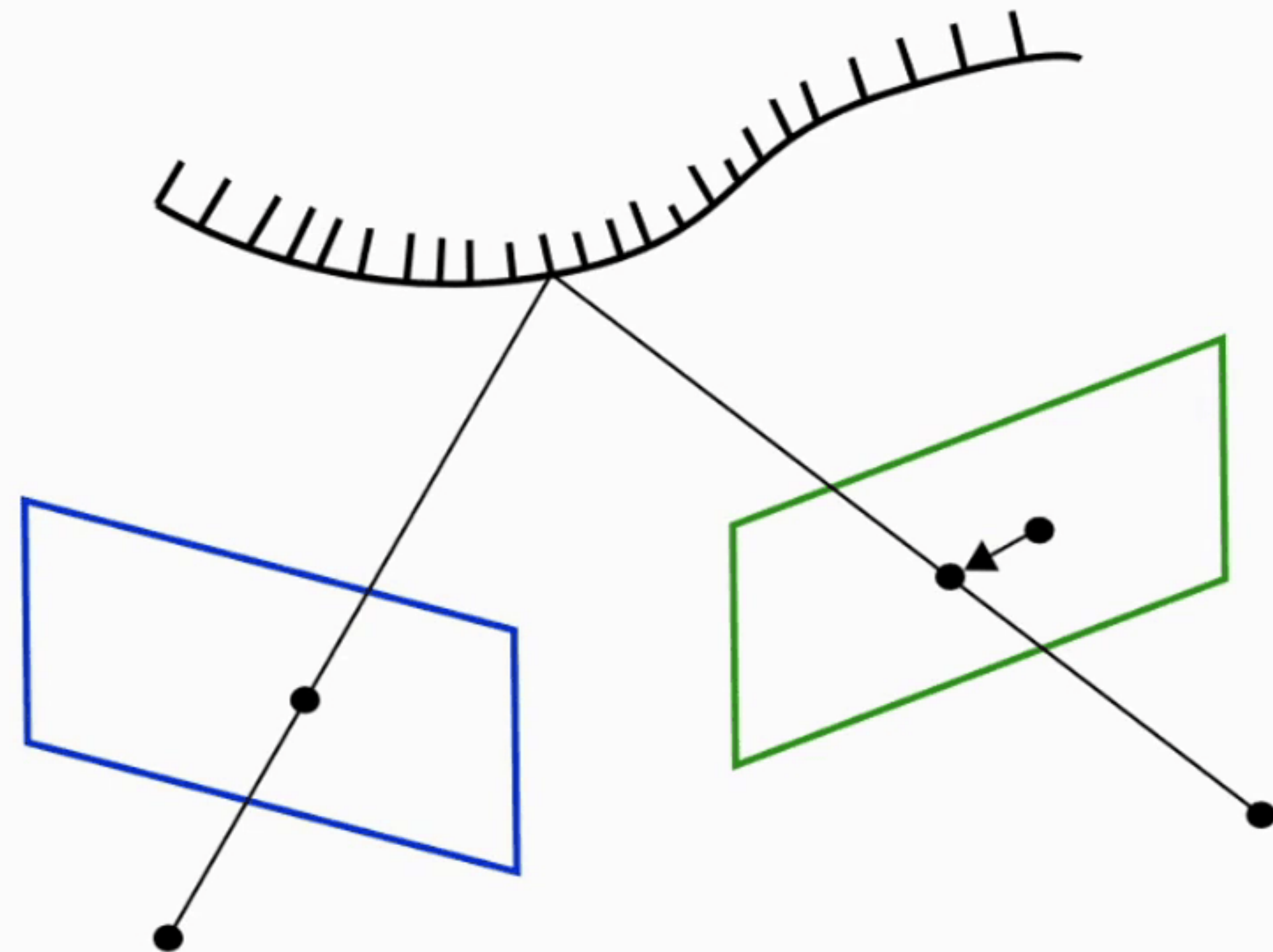
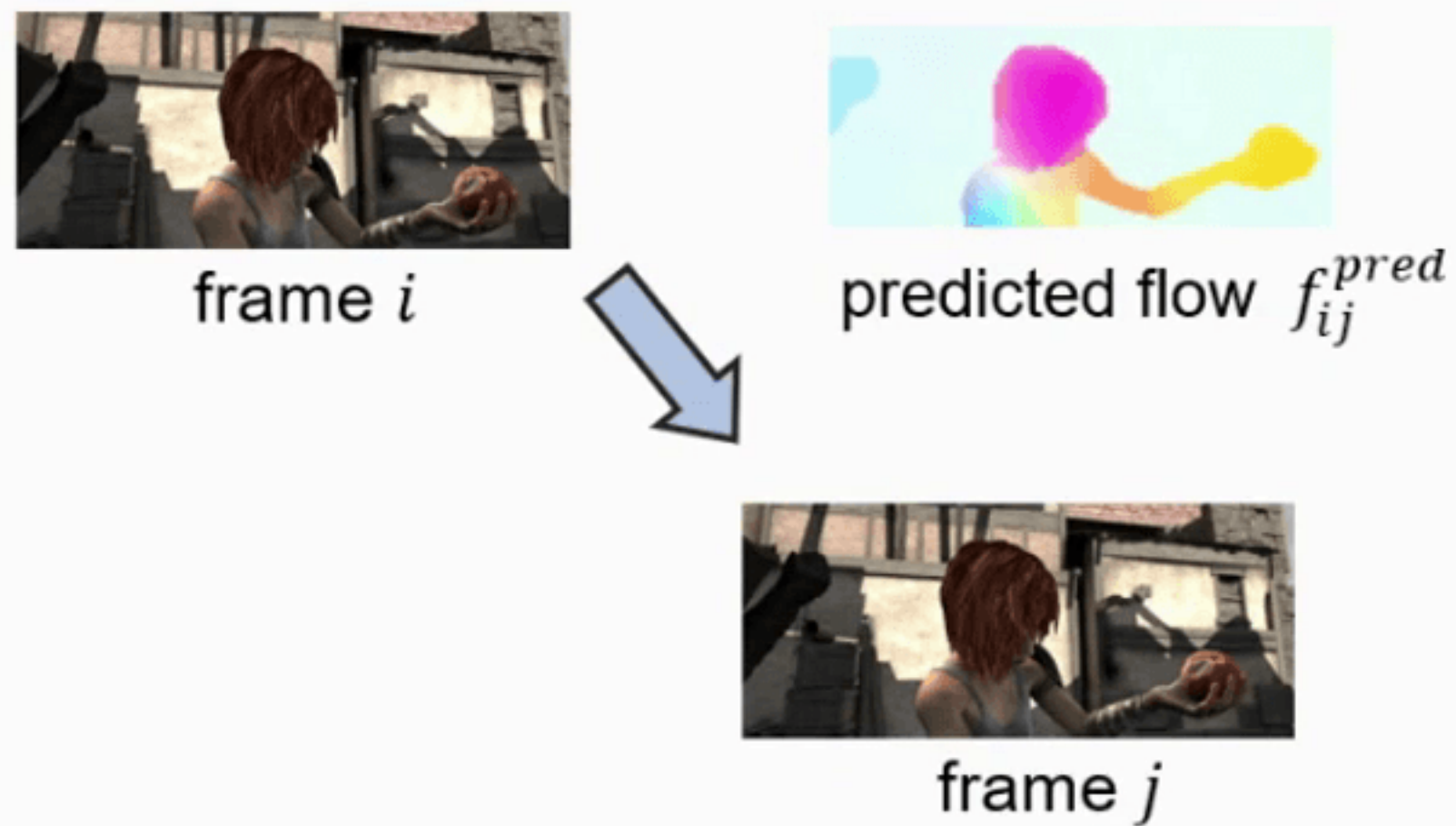
Insufficient Robustness: Frequent and unrecoverable failures

Optical flow is an analytical function of depth and camera poses



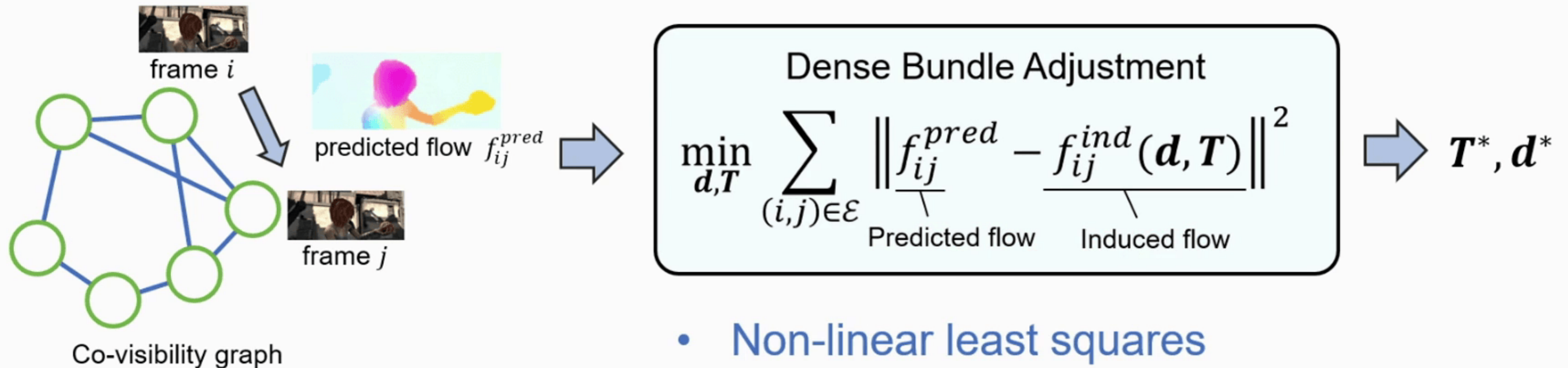
Optical Flow to Depth & Camera Poses

Dense Bundle Adjustment (DBA): find pixelwise depth and poses consistent with predicted flow



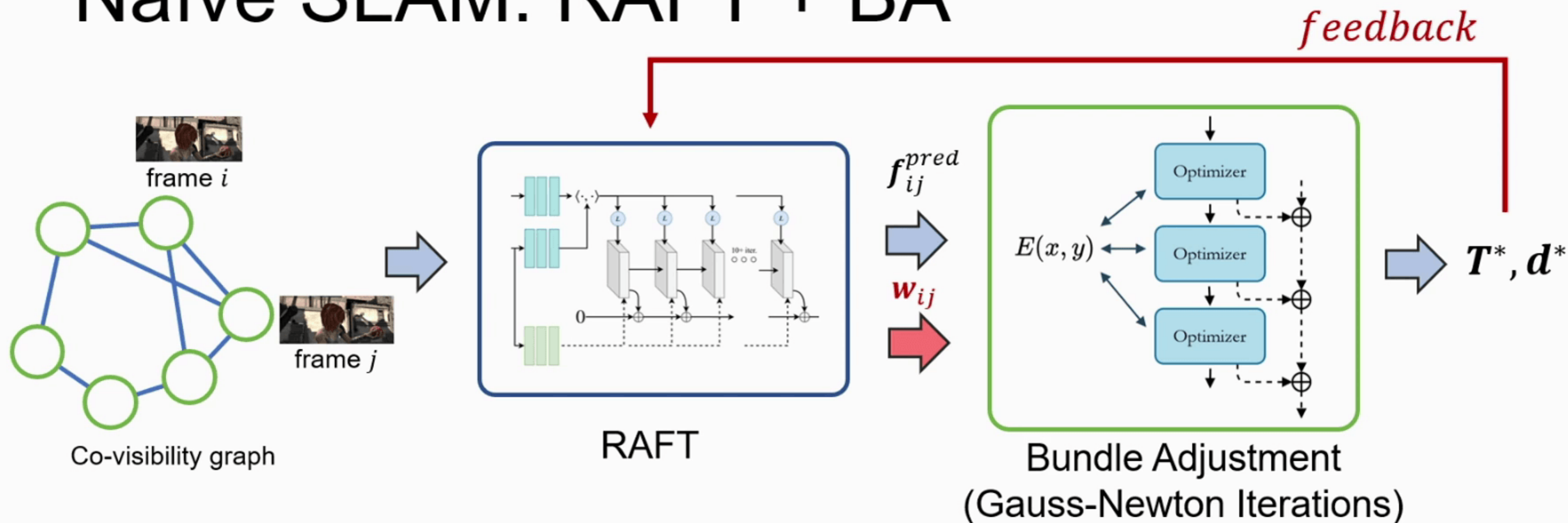
Dense Bundle Adjustment

- **Given:** co-visibility graph $(\mathcal{V}, \mathcal{E})$, predicted flow f_{ij}^{pred}
- **Want:** depth maps $\mathbf{d} = (d_1, \dots, d_i, \dots)$, camera poses $\mathbf{T} = (T_1, \dots, T_i, \dots)$



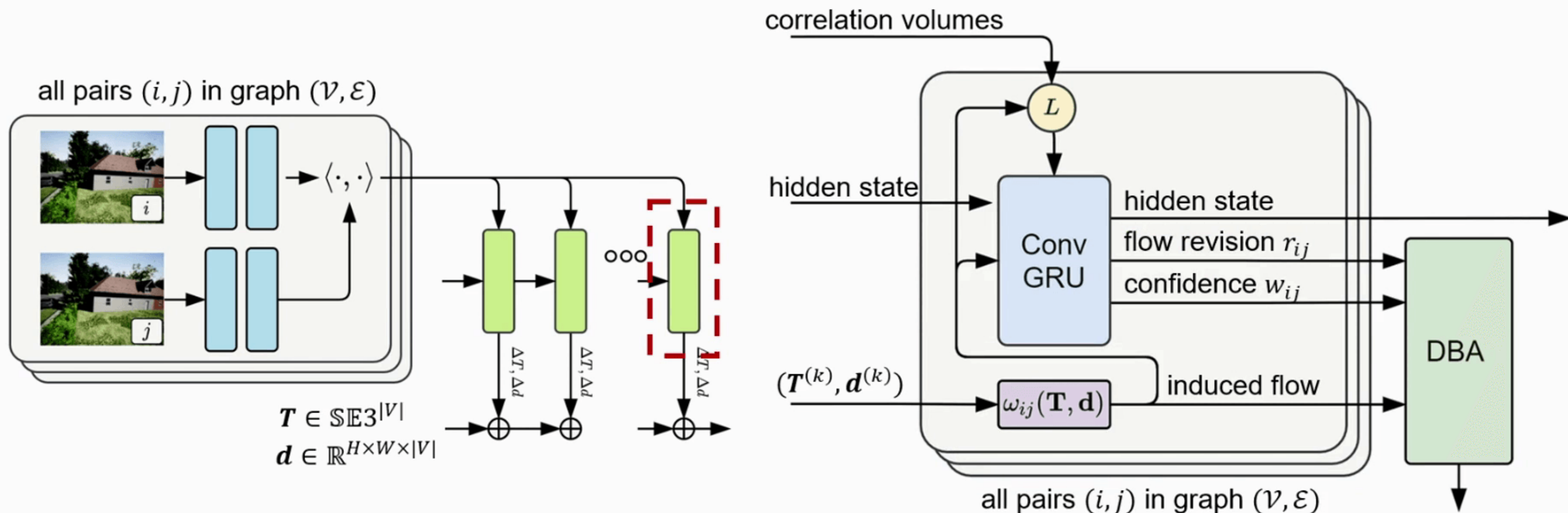
- Non-linear least squares
- Iterative algorithms like Gauss-Newton

Naïve SLAM: RAFT + BA



- Works poorly, because of outliers: visibility, dynamic objects, prediction error
- How to exclude outliers? (1) Predicted Confidence Map (2) Feedback

DROID-SLAM: Architecture

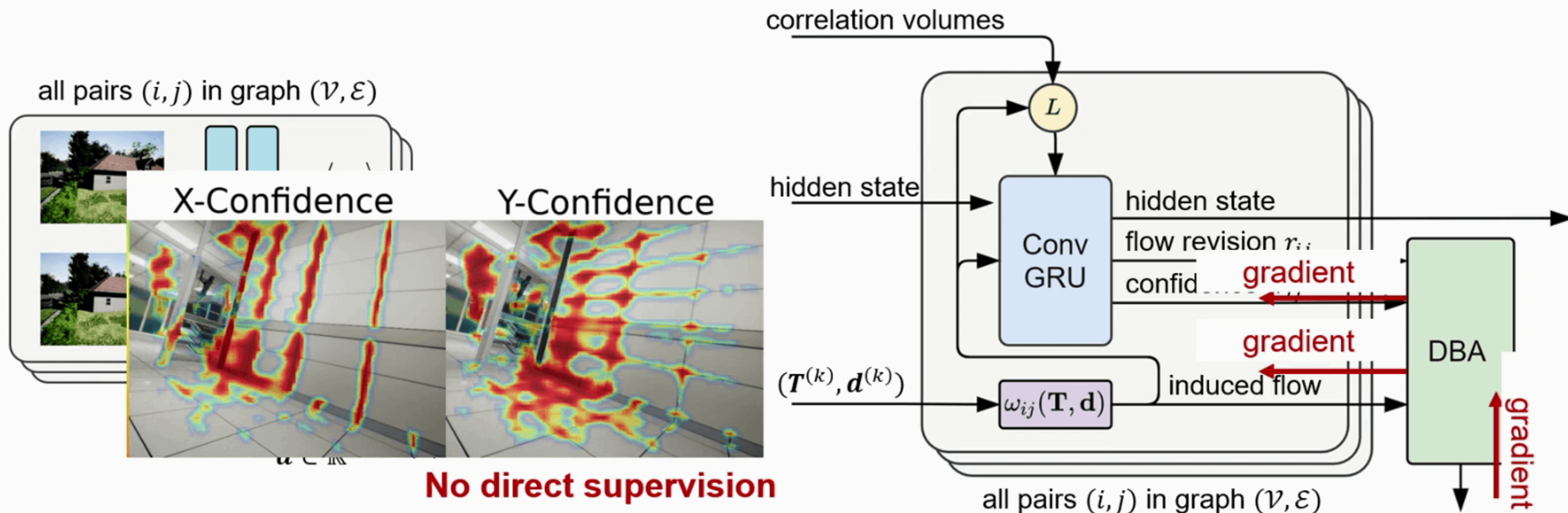


$$\mathbf{x}'_{ij} = \omega_{ij}(\mathbf{T}^{(k)}, \mathbf{d}^{(k)}) + \mathbf{r}_{ij} \quad \text{predicted correspondence (fixed)}$$

$$\mathbf{T}^{(k+1)}, \mathbf{d}^{(k+1)} = \min_{\mathbf{T}, \mathbf{d}} \sum_{(i, j) \in E} \|\omega_{ij}(\mathbf{T}, \mathbf{d}) - \mathbf{x}'_{ij}\|_{diag(w_{ij})}^2$$

Least-Squares: linearize + solve

DROID-SLAM: Architecture



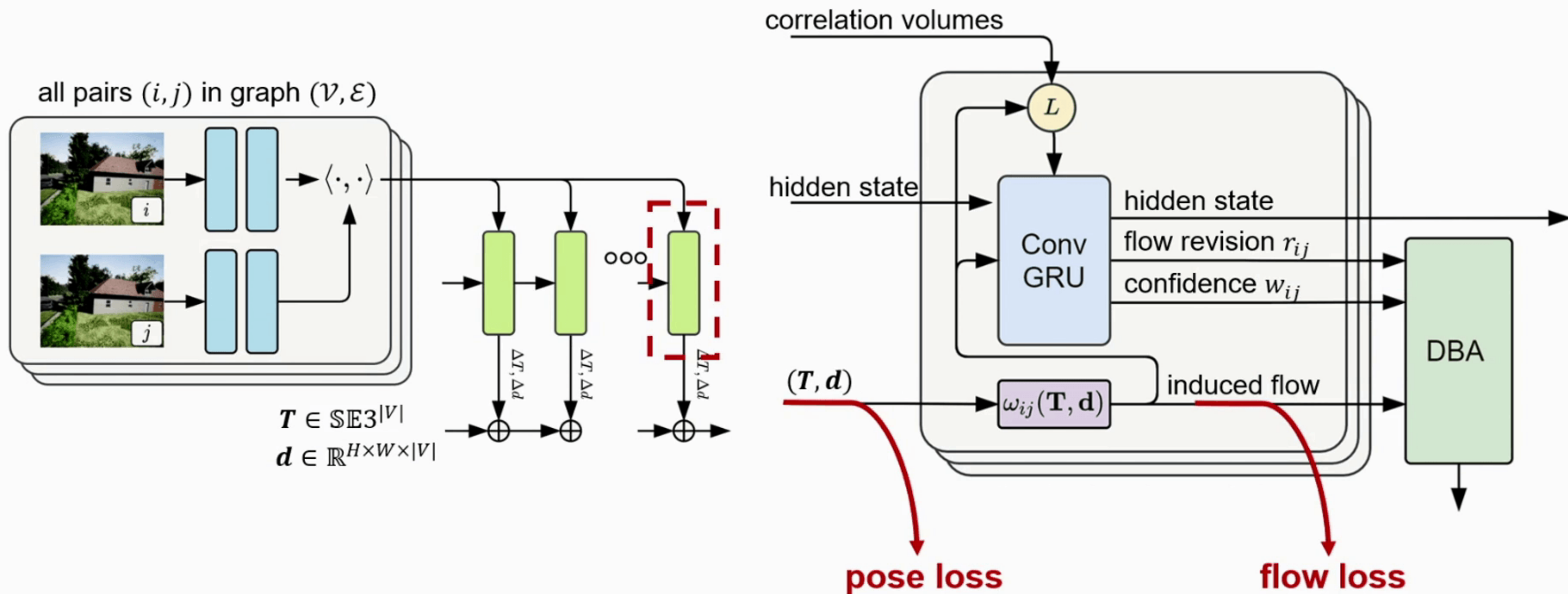
No direct supervision

$$\mathbf{x}'_{ij} = \omega_{ij}(\mathbf{T}^{(k)}, \mathbf{d}^{(k)}) + \mathbf{r}_{ij} \quad \text{predicted correspondence (fixed)}$$

$$\mathbf{T}^{(k+1)}, \mathbf{d}^{(k+1)} = \min_{\mathbf{T}, \mathbf{d}} \sum_{(i,j) \in E} \|\omega_{ij}(\mathbf{T}, \mathbf{d}) - \mathbf{x}'_{ij}\|_{diag(w_{ij})}^2$$

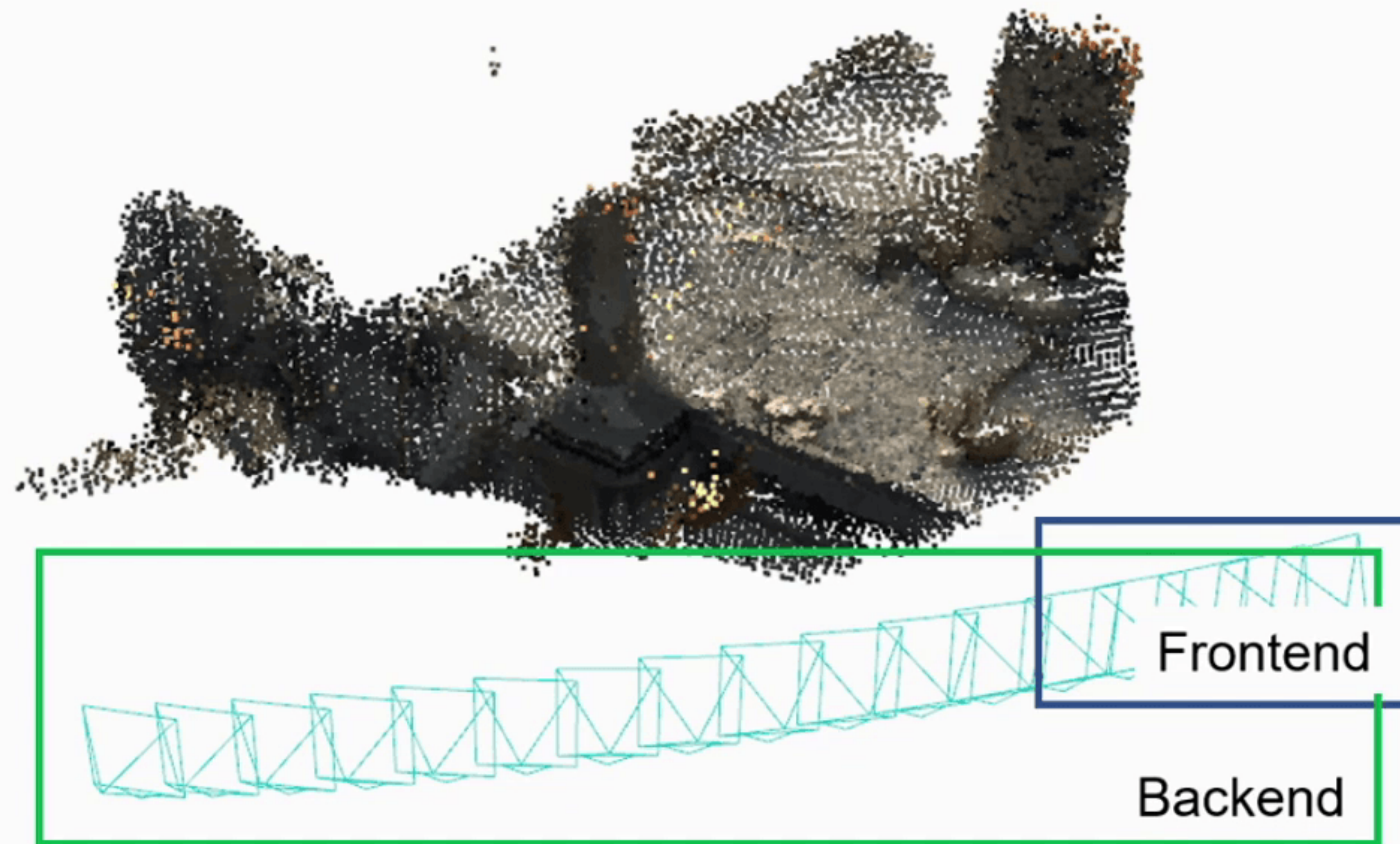
Least-Squares: linearize + solve

Supervision



Inference: Double Window Optimization

- **Frontend:** Optimizes poses/depths in a local window
- **Backend:** Optimizes poses/depths of entire video + loop closure





Video Credit: DroidSLAM. *Teed et. al.*

Questions?